[University Name]

[Faculty / Department]

MASTER'S THESIS

An Effective Object Tracking Algorithm to Visualize the Pedestrian Motion in the Video

Submitted in partial fulfilment of the requirements for the degree of

Master of Science in Computer Science and Technology

Author: [Student Name]

Supervisor: [Supervisor Name]

2026

Abstract

Understanding how pedestrians move through public spaces is important for crowd safety, urban planning, retail analytics, traffic management and security. Surveillance cameras record vast amounts of video, but raw video is difficult for humans to analyse: individual movements are hard to follow in crowded scenes and patterns that emerge over minutes or hours are invisible when watching frame by frame. Converting video into intuitive visual summaries of pedestrian motion — trajectories, flow fields, density heatmaps, dwell-time maps and counts — requires a tracking algorithm that is accurate, keeps identities consistent over long periods and runs efficiently.

This thesis proposes an effective multi-pedestrian tracking algorithm, together with a motion visualization system built upon it. The tracker follows the tracking-by-detection paradigm and makes four improvements targeted at the requirements of motion visualization. (1) A detector fine-tuned for pedestrians in surveillance viewpoints provides high-recall detections, and low-confidence detections are exploited through two-stage association. (2) A motion model that combines a Kalman filter with camera motion compensation and observation-centric re-update reduces drift during occlusion. (3) A fused association cost combining IoU, a lightweight re-identification appearance embedding and a motion-direction consistency term reduces identity switches in crowds. (4) A trajectory post-processing module performs tracklet re-linking, gap interpolation and smoothing to produce complete, clean trajectories suitable for visualization.

On top of the tracker, a visualization module projects trajectories from the image plane to a ground-plane (bird's-eye) view using a homography, and generates trajectory overlays with fading tails, speed- and direction-coloured trajectories, kernel-density occupancy heatmaps, flow-field arrow maps, dwell-time maps and virtual-line counts. Experiments are designed on the MOT17 and MOT20 benchmarks to evaluate tracking accuracy (HOTA, MOTA, IDF1, ID switches) and speed, and on campus and street surveillance videos to evaluate the visualization outputs, including counting accuracy against manual ground truth and a user study of interpretability.

Keywords: pedestrian tracking; multi-object tracking; trajectory visualization; motion analysis; heatmap; homography; re-identification; crowd analysis.

Table of Contents

Abstract **2**

Chapter 1 Introduction **5**

1.1 Research Background 5

1.2 Research Significance 5

1.3 Research Status at Home and Abroad 5

1.4 Research Content 12

1.5 Thesis Organisation 12

Chapter 2 Theoretical Foundations **13**

2.1 Object Detection with Deep Networks 13

2.2 Kalman Filter 13

2.3 Data Association and the Hungarian Algorithm 13

2.4 Person Re-Identification Features 13

2.5 Planar Homography 14

2.6 Kernel Density Estimation 14

2.7 Evaluation Metrics 15

2.8 Chapter Summary 15

Chapter 3 The Proposed Pedestrian Tracking Algorithm **16**

3.1 Overall Framework 16

3.2 Pedestrian Detector 16

3.3 Camera Motion Compensation 17

3.4 Robust Motion Model 18

3.5 Fused Association Cost 21

3.6 Two-Stage Association and Track Management 24

3.7 Algorithm Pseudocode 24

3.8 Chapter Summary 24

Chapter 4 Trajectory Post-Processing and Motion Visualization **26**

4.1 Trajectory Post-Processing 26

4.2 Ground-Plane Projection 29

4.3 Motion Features 30

4.4 Visualization Methods 31

4.5 Chapter Summary 36

Chapter 5 System Design and Implementation 37

5.1 Requirements 37

5.2 Architecture 38

5.3 Implementation Environment 41

5.4 Data Format 41

5.5 User Interface 42

5.6 Chapter Summary 44

Chapter 6 Experiments and Analysis 45

6.1 Datasets 45

6.2 Experimental Environment 45

6.3 Comparison with State-of-the-Art Trackers 45

6.4 Ablation Study 47

6.5 Runtime Analysis 49

6.6 Evaluation of Visualization Outputs 49

6.7 Discussion 54

6.8 Chapter Summary 54

Chapter 7 Conclusion and Future Work 55

7.1 Conclusions 55

7.2 Limitations 55

7.3 Future Work 55

References 56

Acknowledgements 61

Chapter 1
Introduction

1.1 Research Background

Video surveillance systems have been deployed widely in railway stations, airports, shopping centres, campuses and city streets. The volume of video far exceeds what human operators can watch. Intelligent video analysis aims to extract structured information automatically, and the motion of pedestrians is one of the most valuable kinds of information: where people walk, how fast, where they stop, where congestion forms and how flows change over time.

Visualizing pedestrian motion turns this information into images that people can understand at a glance. A trajectory overlay shows individual paths; a heatmap shows where people spend time; a flow map shows dominant directions; a counting chart shows how many people pass a gate per minute. Such visualizations support decisions such as redesigning a station entrance, placing staff, detecting abnormal behaviour or evaluating the effect of a new layout.

All these visualizations depend on multi-object tracking (MOT). Detection alone gives positions in each frame, but without identities it cannot form trajectories, measure speed or avoid counting the same person repeatedly. The quality of the visualization is therefore limited by the quality of the tracker: identity switches break one trajectory into several or merge two people into one; missed detections leave gaps; jittery boxes produce noisy speed estimates. At the same time, the tracker must be fast enough to process long videos or live streams.

1.2 Research Significance

- **Application value:** a reliable pedestrian motion visualization system supports crowd safety management, space design and business analytics, and helps to prevent crowd crush incidents by detecting dangerous densities early.
- **Technical value:** the work identifies which tracker properties matter for visualization (long-term identity consistency and trajectory completeness, not only frame-level accuracy) and improves the tracker accordingly.
- **Integration value:** it provides a complete pipeline from raw video to interpretable visual outputs in real-world coordinates.

1.3 Research Status at Home and Abroad

1.3.1 Pedestrian detection

Hand-crafted detectors. Early pedestrian detection relied on hand-crafted features and shallow classifiers. Dalal and Triggs [1] introduced histograms of oriented gradients (HOG): the image window is divided into small cells, a histogram of gradient orientations is accumulated in each cell, histograms are contrast-normalised over overlapping blocks, and a linear SVM classifies the concatenated descriptor in a sliding-window fashion over an image pyramid. HOG captures the characteristic silhouette of an upright person and remained the reference method for several years. The deformable part model (DPM) of Felzenszwalb et al. [30] extended this idea by representing an object as a coarse root filter plus a set of higher-resolution part filters connected by quadratic deformation costs, trained with a latent SVM. DPM handles articulation and moderate partial occlusion better than a rigid template, but both families depend on sliding windows over many scales and are sensitive to clutter, low resolution and heavy occlusion.

Deep detectors. Convolutional detectors replaced hand-crafted pipelines. Two-stage detectors such as Faster R-CNN [2] generate candidate regions with a region proposal network and then classify and refine each proposal; they are accurate but comparatively slow. One-stage detectors predict boxes densely in a single pass: YOLO [31] formulated detection as regression on a grid, SSD [32] predicted boxes from multiple feature maps with default anchors, and RetinaNet [34] introduced the focal loss to counter the extreme foreground–background imbalance of dense prediction. Anchor-free detectors, such as FCOS [33] and YOLOX [3], remove hand-designed anchors and predict the distances from each location to the box sides, which simplifies training and improves the handling of objects of unusual aspect ratio. Most detectors are pre-trained on COCO [67], whose "person" class covers pedestrians in diverse poses; for tracking, these models are fine-tuned on pedestrian-specific data.

Detection in crowded scenes. Crowds pose two specific problems. First, heavily overlapping pedestrians produce proposals that drift between neighbouring people. Wang et al. proposed repulsion loss [35], which adds to the usual attraction term two repulsion terms that push a predicted box away from other ground-truth boxes and from predictions assigned to other targets, producing tighter, better separated boxes. Second, non-maximum suppression (NMS) with a single IoU threshold cannot distinguish a duplicate detection from a genuinely overlapping second person: a low threshold suppresses true positives in crowds, a high threshold leaves duplicates elsewhere. Soft-NMS [38] decays the scores of overlapping boxes instead of removing them, and Adaptive-NMS [37] predicts a local density for each box and uses it as a box-specific suppression threshold. CrowdDet [36] lets each proposal predict a set of instances (for

example two boxes) trained with an earth mover's distance loss and applies a set-NMS that never suppresses boxes predicted by the same proposal. Benchmarks such as CityPersons [5] and CrowdHuman [4] were created to study occluded and crowded pedestrians; CrowdHuman in particular contains on average more than twenty persons per image with full-body, visible-body and head annotations, and is the standard additional training set for MOT detectors.

1.3.2 Multi-object tracking

Classical approaches. Multi-object tracking (MOT) was traditionally formulated as a data association problem. Multiple hypothesis tracking (MHT) [40] maintains a tree of association hypotheses over time and defers decisions until more evidence is available; it is principled but its hypothesis space grows exponentially and must be pruned aggressively. Global (batch) methods formulate association over a whole sequence as a graph problem; Zhang et al. [39] modelled tracks as flows in a network whose nodes are detections and solved for the globally optimal set of trajectories with min-cost flow. Such offline methods exploit future information and can bridge long occlusions, but they are not suited to live operation.

The SORT family. SORT [6] showed that, given a strong detector, a simple online tracker composed of a constant-velocity Kalman filter [58] and Hungarian assignment [59] on IoU distance can reach competitive accuracy at very high speed. Its weakness is identity switching under occlusion, since IoU alone cannot recover a target after it disappears. DeepSORT [7] added an appearance descriptor from a re-identification CNN, a gallery of recent features per track, Mahalanobis gating using the Kalman covariance and a matching cascade that gives priority to recently seen tracks. ByteTrack [10] observed that low-confidence detections often correspond to occluded but real pedestrians and proposed to associate every detection box: high-score boxes are matched first, and the remaining tracks are then matched to low-score boxes by IoU, which recovers many otherwise lost targets while discarding low-score background boxes that do not match any track. OC-SORT [11] analysed the failure of the Kalman filter during occlusion — without observations its velocity estimate drifts and the error accumulates quadratically with time — and introduced observation-centric re-update, observation-centric momentum (a direction-consistency term in the cost) and observation-centric recovery. BoT-SORT [12] combined ByteTrack with camera motion compensation by global motion estimation, a Kalman state that predicts width and height directly, and an IoU–ReID fusion that takes the minimum of the two distances after gating. StrongSORT [13] revisited DeepSORT with a stronger detector and embedding, an exponential moving average of features, camera motion compensation and NSA Kalman filtering, and proposed two light-weight post-processing modules: AFLink, an

appearance-free link model for joining tracklets, and Gaussian-smoothed interpolation (GSI). Deep OC-SORT [14] added appearance to OC-SORT with a dynamic appearance weight that depends on detection confidence and an adaptive weighting that boosts the appearance cost when the embedding is discriminative.

Joint detection and tracking. Instead of separate models, JDE [8] attached an embedding head to a one-stage detector so that boxes and appearance features are produced in one forward pass. FairMOT [9] argued that anchor-based detection and re-identification compete for features and proposed an anchor-free, high-resolution centre-based design with balanced detection and ReID branches. Tracktor [41] converted a detector into a tracker by regressing the previous box positions in the current frame, and CenterTrack [42] predicted object centres together with offsets to the previous frame. These methods reduce computation, but the shared backbone typically yields weaker embeddings than a dedicated ReID network.

Transformer trackers. Transformer-based methods model tracking with queries. TransTrack [43] uses object queries for detection and track queries from the previous frame for association; TrackFormer [16] and MOTR [17] propagate track queries through time so that identity is carried implicitly by the query, removing hand-designed association. These end-to-end trackers are elegant, but early versions struggled with detection of newborn objects in crowds; MOTRv2 [68] therefore injected proposals from a separate YOLOX detector. Their computational cost remains considerably higher than that of SORT-style trackers, which limits their use in long-duration and multi-stream surveillance.

1.3.3 Person re-identification

Person re-identification (ReID) learns embeddings in which crops of the same person are close. A strong baseline by Luo et al. [44] (often called BoT, "bag of tricks") showed that a ResNet-50 with a carefully chosen set of training tricks — warm-up learning rate, random erasing, label smoothing, last-stride removal, a BNNeck that separates the feature used by the triplet loss [46] from that used by the classification loss, and centre loss — rivals much more complex architectures. OSNet [15] introduced omni-scale feature learning: each residual block contains several streams with different receptive fields, fused by a unified aggregation gate that produces channel-wise weights, and depthwise-separable convolutions keep the model very small; the x0.25 variant has well under a million parameters, which makes it attractive for real-time tracking. TransReID [45] applied a vision transformer with a jigsaw patch module that shuffles and regroups patch tokens to learn robust local features, and side-information embeddings that encode camera and viewpoint. For MOT, embedding quality must be traded against cost,

because the ReID network is run on every detected person.

1.3.4 Crowd analysis

Crowd counting. When crowds are too dense for individual detection, counting is formulated as density map regression. MCNN [47] used three columns of convolutional filters with different receptive fields to handle scale variation, and produced density maps whose integral equals the count; ground truth maps are generated by placing geometry-adaptive Gaussian kernels on annotated heads. CSRNet [48] replaced the multi-column design with a VGG-16 front end followed by dilated convolutions, enlarging the receptive field without reducing resolution. Density-based counting is robust in extremely crowded scenes but yields no identities and hence no trajectories.

Pedestrian dynamics. The social force model of Helbing and Molnár [49] describes each pedestrian as a particle driven by a desired velocity towards a goal, plus repulsive forces from other pedestrians and obstacles and attractive forces from friends or points of interest:

$$m_i \, dv_i/dt = m_i \, (v_i^0 \, e_i - v_i) / \tau_i + \Sigma_{j \neq i} f_{ij} + \Sigma_W f_{iW} \qquad (1.1)$$

where v_i^0 is the desired speed, e_i the desired direction, τ_i a relaxation time, f_{ij} the interaction force with pedestrian j and f_{iW} the force from walls. The model reproduces lane formation in counter-flows and clogging at bottlenecks and is the conceptual basis of many simulators. It also motivates the direction-consistency term in this thesis: over short horizons, pedestrians move smoothly towards their goals and rarely reverse abruptly.

Abnormal behaviour detection. Mehran et al. [50] estimated interaction forces from optical-flow particles using the social force model and detected abnormal crowd behaviour (for example panic dispersal) with a bag-of-words model of force flows. Mahadevan et al. [51] introduced the UCSD anomaly dataset and modelled normal crowd appearance and dynamics with mixtures of dynamic textures, flagging temporal and spatial outliers. These methods operate on low-level motion and do not require tracking, but their outputs are hard to relate to individual paths, whereas trajectory-based features such as counter-flow, loitering and running are directly interpretable.

1.3.5 Trajectory prediction

Trajectory prediction models future pedestrian paths from past observations, and its benchmarks (ETH [64] and UCY) are built from tracked trajectories in ground-plane coordinates. Social LSTM [22] models each person with an LSTM and introduces a social pooling layer that aggregates the hidden states of neighbours in a spatial grid. Social GAN [23] trains a generator–discriminator pair with a variety loss to produce

multiple socially acceptable futures. Trajectron++ [52] represents the scene as a spatio-temporal graph, uses a conditional variational autoencoder to produce multimodal distributions and can incorporate agent dynamics and semantic maps. These models assume clean, complete, metric trajectories as input — exactly what a tracking-plus-projection pipeline such as the one proposed here must provide — and identity switches or perspective-distorted coordinates would directly corrupt their training data.

1.3.6 Visual analytics of movement

The visualization of movement has a long history in geography and visual analytics [24]. Hägerstrand's time geography [53] introduced the space-time path, and the space-time cube [54] displays trajectories in a 3-D volume with two spatial axes and time as the vertical axis, making co-location and waiting visible as vertical segments, at the cost of occlusion and difficult 3-D interaction. Flow maps aggregate movements between origins and destinations into arrows whose width is proportional to volume; edge bundling [66] reduces clutter when many trajectories overlap by attracting compatible edges towards each other. Density-based methods apply kernel density estimation [55] to positions or trajectory segments to produce continuous heatmaps; the choice of kernel bandwidth strongly affects the result, trading smoothness against detail. Andrienko et al. [24] systematised these techniques into views of movers (individual trajectories), of spatial events (stops, crossings) and of spatial situations (aggregated flows per area and time). Commercial video analytics products offer heatmaps and people counting, but they are proprietary and do not report how tracking errors propagate to their visual outputs.

1.3.7 Comparison of representative methods

Method	Category	Motion cue	Appearance cue	Occlusion handling	Real-time	Output for visualization
SORT [6]	Tracking-by-detection	Kalman, IoU	None	Weak	Yes	Fragmented tracks
DeepSORT [7]	Tracking-by-detection	Kalman, Mahalanobis	Separate CNN	Cascade, gallery	Moderate	Tracks, image plane
FairMOT [9]	Joint detection–embedding	Kalman, IoU	Shared head	Moderate	Yes	Tracks, image plane
ByteTrack [10]	Tracking-by-detection	Kalman, IoU	None	Low-score boxes	Yes	Tracks, image plane

Method	Category	Motion cue	Appearance cue	Occlusion handling	Real-time	Output for visualization
OC-SORT [11]	Tracking-by-detection	Kalman with OC re-update	None	Re-update, recovery	Yes	Tracks, image plane
BoT-SORT [12]	Tracking-by-detection	Kalman with CMC	Separate CNN	IoU–ReID fusion	Moderate	Tracks, image plane
StrongSORT [13]	Tracking-by-detection	NSA Kalman, CMC	Separate CNN, EMA	AFLink, GSI	Moderate	Post-processed tracks
MOTR [17]	Transformer	Implicit (queries)	Implicit	Query propagation	No	Tracks, image plane
CSRNet [48]	Density regression	None	None	Not applicable	Yes	Density map, no identity
PMV-Track (this thesis)	Tracking-by-detection	Kalman, CMC, OC re-update, direction	Adaptive lightweight ReID	Two-stage, re-linking, GP interpolation	Yes (target)	Metric trajectories and visual summaries

Table 1.1 Comparison of representative methods with respect to the requirements of motion visualization.

1.3.8 Gap analysis

From the above review, five gaps can be identified that motivate this thesis.

- **Metric orientation.** MOT research optimises benchmark metrics such as MOTA and HOTA, but the effect of tracking errors on downstream visualizations — broken trajectories, phantom stops, double counts — is rarely analysed or measured.
- **Long-term completeness.** Online trackers produce fragmented trajectories under long occlusion. Post-processing modules (AFLink, GSI) exist, but they are designed for benchmark submission and operate in image coordinates without physical constraints such as maximum walking speed.
- **Cost of appearance.** Appearance features improve identity consistency, but running a ReID network on every detection in every frame doubles the computational cost even in sparse scenes where IoU alone suffices.
- **Perspective distortion.** Visualizations are often produced in image coordinates, where distant pedestrians appear to move slowly and occupy small areas, misrepresenting density, speed and dwell time; metric ground-plane analysis is common in trajectory prediction datasets but rarely integrated with online tracking.

- **Fragmented tooling.** Detection, tracking, calibration and visualization are usually separate tools; there are few open, integrated pipelines that go from raw surveillance video to interpretable, metric visual summaries with documented error behaviour.

1.4 Research Content

- Design an effective pedestrian tracking algorithm with improved detection, motion modelling, fused association and trajectory post-processing.
- Design a ground-plane projection method so that motion statistics are measured in real-world units.
- Design a set of motion visualizations: trajectories, heatmaps, flow fields, dwell time and counting.
- Implement an integrated system and evaluate tracking accuracy, efficiency and visualization quality.

1.5 Thesis Organisation

Chapter 2 introduces the related theory. Chapter 3 presents the proposed tracking algorithm. Chapter 4 presents trajectory post-processing and the visualization methods. Chapter 5 describes the system design. Chapter 6 presents experiments. Chapter 7 concludes the thesis.

Chapter 2
Theoretical Foundations

2.1 Object Detection with Deep Networks

A one-stage detector such as YOLOX consists of a backbone (CSPDarknet), a neck (PAFPN) and a decoupled anchor-free head that predicts, at each location, the box, an objectness score and class scores. For pedestrian tracking only the person class is used. The detector outputs, for each frame t, a set of boxes $b = (x_1, y_1, x_2, y_2)$ with confidence scores s.

2.2 Kalman Filter

The Kalman filter estimates the state of a linear dynamic system from noisy observations. With the state $x = [x_c, y_c, w, h, x'_c, y'_c, w', h']^T$ and a constant-velocity model, prediction and update are:

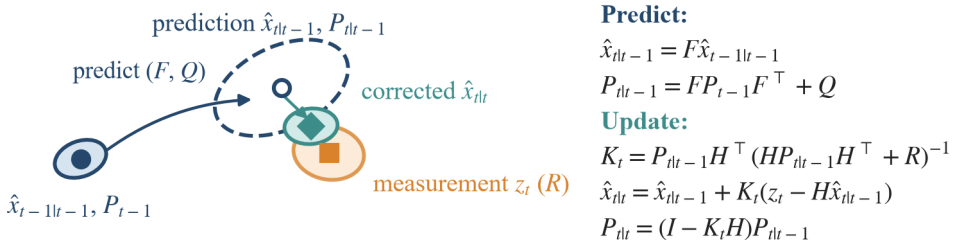


Figure 2.1 Kalman filter predict–update cycle: the prediction increases uncertainty, and fusing it with the measurement gives a corrected estimate with reduced uncertainty.

$$\hat{x}_{k|k-1} = F\,\hat{x}_{k-1},\; P_{k|k-1} = F\,P_{k-1}\,F^T + Q \tag{2.1}$$

$$K_k = P_{k|k-1}\,H^T\,(H\,P_{k|k-1}\,H^T + R)^{-1} \tag{2.2}$$

$$\hat{x}_k = \hat{x}_{k|k-1} + K_k(z_k - H\,\hat{x}_{k|k-1}) \tag{2.3}$$

Predicting width and height directly (rather than area and aspect ratio) has been found to fit the box shape more accurately for pedestrians.

2.3 Data Association and the Hungarian Algorithm

Given a cost matrix $C \in R^{m\times n}$ between m tracks and n detections, association solves:

$$min\ \Sigma_i \Sigma_j\, C_{ij}\, a_{ij}\ s.t.\ \Sigma_j\, a_{ij} \le 1,\ \Sigma_i\, a_{ij} \le 1,\ a_{ij} \in \{0, 1\} \tag{2.4}$$

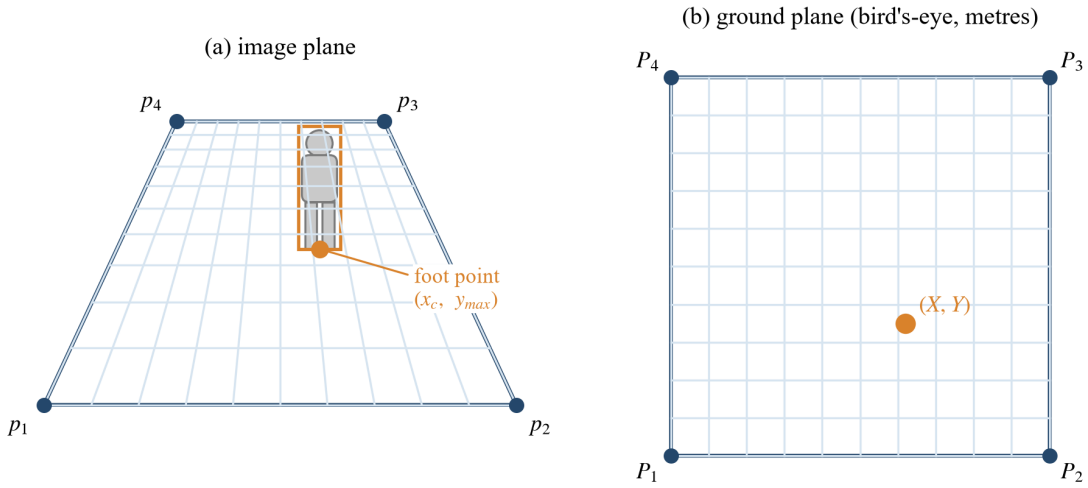
which is solved optimally by the Hungarian (Kuhn–Munkres) algorithm or by the Jonker–Volgenant algorithm (LAPJV), which is faster in practice.

2.4 Person Re-Identification Features

Person re-identification (ReID) networks such as OSNet map a pedestrian crop to an embedding vector f with ||f|| = 1, trained so that crops of the same person are close and those of different people are far apart. The appearance distance between a track and a detection is the cosine distance $d_{app} = 1 - f_{track} \cdot f_{det}$. To make track features robust, an exponential moving average is used: $f_{track} \leftarrow \alpha f_{track} + (1 - \alpha) f_{det}$.

2.5 Planar Homography

Pedestrians walk on an approximately planar ground. A point (u, v) in the image and its ground-plane coordinate (X, Y) are related by a 3 × 3 homography H:



$[X, Y, 1]^\top \sim H^{-1}[x, y, 1]^\top$, H estimated from 4+ point pairs (DLT)

Figure 2.2 Homography between the image plane and the bird's-eye ground plane from four point correspondences; the foot point of a bounding box is mapped to ground coordinates.

$$[X, Y, 1]^T \propto H\,[u, v, 1]^T \tag{2.5}$$

H has 8 degrees of freedom and can be estimated from at least four point correspondences (e.g., floor tile corners or road markings with known dimensions) by the direct linear transform. The foot point of each pedestrian — the bottom centre of the box — is used as the image point, since it lies on the ground plane.

2.6 Kernel Density Estimation

A density heatmap is produced by kernel density estimation over ground-plane positions p_i:

$$D(x) = (1 / (N h^2)) \Sigma_i K((x - p_i) / h) \tag{2.6}$$

with a Gaussian kernel K and bandwidth h, chosen by Silverman's rule or set to a physically meaningful value (e.g., 0.5 m).

2.7 Evaluation Metrics

MOTA, IDF1 and HOTA (with DetA and AssA) measure tracking quality; ID switches (IDSW) and fragmentations (Frag) count identity errors; FPS measures speed. For visualization, counting error and trajectory completeness are additionally used (Chapter 6).

CLEAR MOT. MOTA [62] aggregates three error types over all frames t, with GT_t the number of ground-truth objects:

$$MOTA = 1 - \Sigma_t (FN_t + FP_t + IDSW_t) / \Sigma_t GT_t \tag{2.7}$$

Because false negatives and false positives are usually far more numerous than identity switches, MOTA is dominated by detection quality and is relatively insensitive to association errors, which are the errors most harmful to trajectory visualization.

IDF1. IDF1 [63] computes a one-to-one matching between ground-truth and predicted trajectories over the whole sequence that maximises the number of identity-consistent matched detections (IDTP), and reports

$$IDF1 = 2\ IDTP / (2\ IDTP + IDFP + IDFN) \tag{2.8}$$

It therefore rewards long-term identity consistency and is closely related to the completeness of trajectories.

HOTA. HOTA [18] balances detection and association. For a localisation threshold α, detection accuracy is $DetA_\alpha = |TP| / (|TP| + |FN| + |FP|)$, and association accuracy averages, over true positives c, the overlap between the predicted and ground-truth trajectories that c belongs to:

$$AssA_\alpha = (1/|TP|) \Sigma_{c \in TP} |TPA(c)| / (|TPA(c)| + |FNA(c)| + |FPA(c)|),$$
$$HOTA_\alpha = sqrt(DetA_\alpha \cdot AssA_\alpha) \tag{2.9}$$

and the final HOTA is the average of $HOTA_\alpha$ over α = 0.05, 0.10, ..., 0.95. Reporting DetA and AssA separately shows whether an improvement comes from detection or from association.

2.8 Chapter Summary

This chapter introduced detection, the Kalman filter, data association, re-identification features, homography and kernel density estimation, which are the theoretical tools used in this thesis.

Chapter 3
The Proposed Pedestrian Tracking Algorithm

3.1 Overall Framework

The proposed tracker, called PMV-Track (Pedestrian Motion Visualization Tracker), processes each frame in the following steps (Figure 3.1): (1) pedestrian detection; (2) camera motion compensation; (3) Kalman prediction; (4) first-stage association of high-score detections using the fused cost; (5) second-stage association of low-score detections using IoU; (6) track management with observation-centric re-update; (7) output of active tracks. After the whole video (or a sliding window, in online mode) is processed, trajectory post-processing is applied.

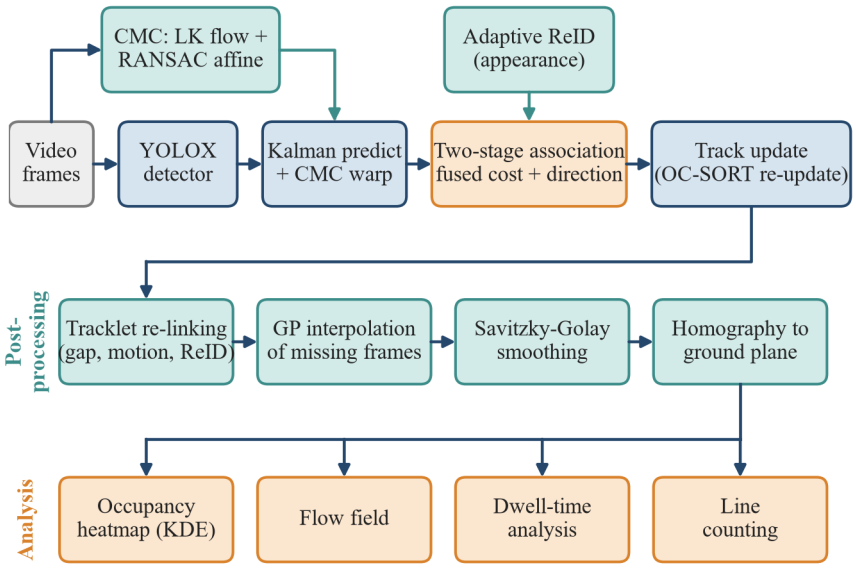


Figure 3.1 PMV-Track pipeline: YOLOX detection, camera motion compensation, Kalman prediction, two-stage association with fused cost and adaptive ReID, OC-SORT update, trajectory post-processing and motion analysis.

3.2 Pedestrian Detector

YOLOX is selected as the detector because of its good accuracy–speed balance and its wide use in MOT. It is initialised from COCO weights and fine-tuned on a mixture of CrowdHuman and the MOT17 training set (half split) with person as the only class. Surveillance cameras often look down at pedestrians from an elevated angle; to adapt to this viewpoint, training data are augmented with random perspective transformations and scale jitter. Detections are kept down to a low score of 0.1, so that occluded pedestrians can be recovered by the second-stage association.

3.3 Camera Motion Compensation

Although surveillance cameras are usually static, pan-tilt-zoom cameras and handheld or vehicle-mounted cameras are also common. Camera motion invalidates the constant-velocity assumption. Following BoT-SORT, sparse feature points (e.g., Shi–Tomasi corners [61]) outside detected boxes are tracked between consecutive frames with pyramidal Lucas–Kanade optical flow [27], and an affine transformation A_t is estimated by RANSAC [28]. The predicted state and covariance of every track are then warped by A_t. For static cameras, the estimated transformation is close to identity and the step can be disabled to save time.

3.3.1 Feature extraction and matching

At frame t−1, the image is downscaled by a factor of two and converted to grey scale. A binary mask excludes all detected pedestrian boxes (enlarged by 10 %), since pedestrians move independently of the background and would bias the estimate. Up to 1000 Shi–Tomasi corners with minimum quality 0.01 and minimum distance 1 pixel are extracted inside the mask. Each corner is tracked into frame t with a three-level pyramidal Lucas–Kanade tracker using 21 × 21 windows, and correspondences whose forward–backward error exceeds one pixel are discarded. Coordinates are rescaled to full resolution before estimation.

3.3.2 Affine estimation with RANSAC

The global motion between frames is modelled by a 2-D affine transformation, which covers translation, rotation, scaling and shear and is adequate for the small inter-frame motion of a camera observing a distant scene:

$$[u', v']^T = M\,[u, v]^T + T,\ M = [\ a_{11}\ a_{12}\ ;\ a_{21}\ a_{22}\ ],\ T = [\ t_x,\ t_y\ ]^T \tag{3.1}$$

The six parameters are linear in the correspondences. Each correspondence contributes two equations:

$$[\ u\ v\ 1\ 0\ 0\ 0\ ;\ 0\ 0\ 0\ u\ v\ 1\ ]\ [\ a_{11},\ a_{12},\ t_x,\ a_{21},\ a_{22},\ t_y\ ]^T = [\ u',\ v'\ ]^T \tag{3.2}$$

so three non-collinear correspondences determine the transformation. RANSAC repeatedly draws a minimal sample of three correspondences, computes the affine parameters, and counts inliers whose transfer error $\|A(u) - u'\|$ is below a threshold ε (3 pixels). The number of iterations required to draw at least one outlier-free sample with probability p, given an inlier ratio w, is

$$N = \log(1 - p) / \log(1 - w^3) \tag{3.3}$$

which for p = 0.99 and w = 0.5 gives about 35 iterations. The model with the largest consensus set is refined by linear least squares over all its inliers. If fewer than 10 inliers

are found (for example in a textureless scene), the identity transformation is used.

3.3.3 Warping the Kalman state and covariance

Let $M \in R^{2\times2}$ and $T \in R^2$ be the estimated affine parts. The state contains four position-like components (x_c, y_c, w, h) and four velocity components. The centre is transformed by the full affine map, while width, height and all velocities are transformed only by the linear part. Writing the 8 × 8 block-diagonal matrix

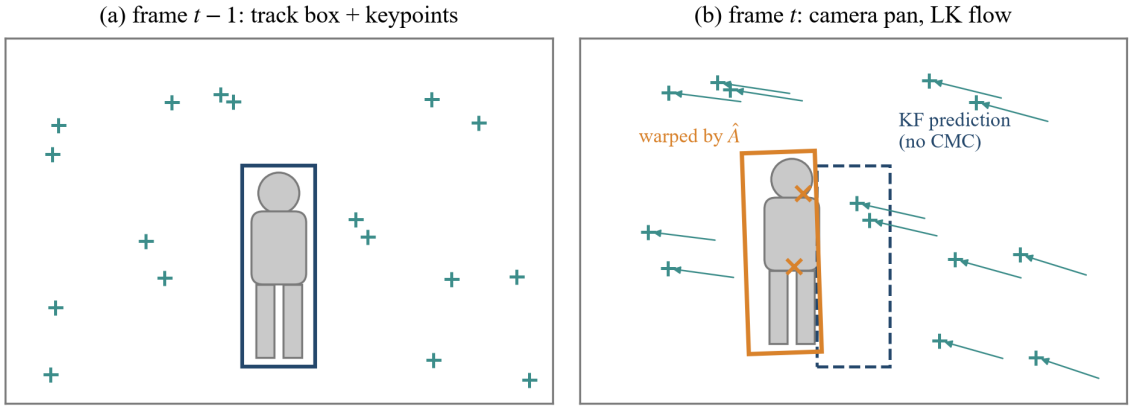


Figure 3.2 Camera motion compensation: background keypoints are tracked with LK optical flow, an affine transform is estimated with RANSAC (moving-person outliers rejected), and the predicted boxes are warped accordingly.

$$M_8 = diag(M, M, M, M) \tag{3.4}$$

and the translation vector $T_8 = [\,t_x, t_y, 0, 0, 0, 0, 0, 0\,]^T$, the predicted state and covariance are corrected as

$$x'_{k|k-1} = M_8\,\hat{x}_{k|k-1} + T_8,\quad P'_{k|k-1} = M_8\,P_{k|k-1}\,M_8^T \tag{3.5}$$

Applying M to the (w, h) pair is an approximation that is exact for pure scaling and translation, which dominate PTZ and slow handheld motion. The covariance transformation follows from the linear propagation of uncertainty; the translation does not change the covariance. The last observations stored for OC-SORT re-update are warped by the same transformation so that virtual trajectories are generated in the current camera frame.

3.4 Robust Motion Model

3.4.1 Width–height Kalman state

The Kalman state uses the box centre, width and height and their velocities, $x = [x_c, y_c, w, h, x'_c, y'_c, w', h']^T$, and the measurement is $z = [x_c, y_c, w, h]^T$. With a

constant-velocity model and a time step of one frame, the transition and observation matrices are

$$F = [\, I_4 \; I_4 \,;\; 0_4 \; I_4 \,],\; H = [\, I_4 \; 0_4 \,] \tag{3.6}$$

where I_4 is the 4 × 4 identity and 0_4 the 4 × 4 zero matrix. Written out in full, F is the 8 × 8 matrix shown in Table 3.1, and H selects the first four components of the state.

Row	$\mathbf{x_c}$	$\mathbf{y_c}$	**w**	**h**	$\mathbf{x'_c}$	$\mathbf{y'_c}$	**w'**	**h'**
x_c	1	0	0	0	1	0	0	0
y_c	0	1	0	0	0	1	0	0
w	0	0	1	0	0	0	1	0
h	0	0	0	1	0	0	0	1
x'_c	0	0	0	0	1	0	0	0
y'_c	0	0	0	0	0	1	0	0
w'	0	0	0	0	0	0	1	0
h'	0	0	0	0	0	0	0	1

Table 3.1 State transition matrix F of the constant-velocity width–height Kalman filter.

Process and measurement noise are scaled by the box height, so that near (large) and far (small) pedestrians are treated consistently. Following the convention of DeepSORT and BoT-SORT, with weights $\sigma_p = 1/20$ for positions and $\sigma_v = 1/160$ for velocities, and with $\hat{h}$ the current height estimate, the noise covariances are diagonal:

$$Q_k = diag(\, (\sigma_p \hat{h})^2,\, (\sigma_p \hat{h})^2,\, (\sigma_p \hat{h})^2,\, (\sigma_p \hat{h})^2,\, (\sigma_v \hat{h})^2,\, (\sigma_v \hat{h})^2,\, (\sigma_v \hat{h})^2,\, (\sigma_v \hat{h})^2 \,) \tag{3.7}$$

$$R_k = diag(\, (\sigma_p h_k)^2,\, (\sigma_p h_k)^2,\, (\sigma_p h_k)^2,\, (\sigma_p h_k)^2 \,) \tag{3.8}$$

where h_k is the height of the measured box. Using the same noise for width and height (rather than a fixed aspect ratio) allows the filter to follow the shape changes of a walking person, whose apparent width oscillates with the gait cycle. A newly created track is initialised with zero velocity and covariance $P_0 = \mathrm{diag}(\, (2\sigma_p h)^2\, I_4,\, (10\sigma_v h)^2\, I_4 \,)$, reflecting high initial uncertainty in velocity. To prevent the width or height from becoming negative during long extrapolation, the predicted velocity of w and h is set to zero whenever the predicted value would fall below one pixel.

3.4.2 Mahalanobis gating

The innovation covariance $S = H\, P_{k|k-1}\, H^T + R$ defines the expected spread of the measurement. The squared Mahalanobis distance between a track and a detection z_j,

$$d_M^{\,2}(i, j) = (\, z_j - H \hat{x_i} \,)^T\, S_i^{\,-1}\, (\, z_j - H \hat{x_i} \,) \tag{3.9}$$

follows a χ^2 distribution with four degrees of freedom when the association is correct. Pairs with d_M^2 above the 95 % quantile (9.4877) are considered motion-inconsistent. This test complements the IoU gate: IoU is scale-dependent and becomes zero for small, fast-moving targets, while the Mahalanobis gate adapts to the uncertainty of each track and widens automatically while a track is lost.

3.4.3 Observation-centric re-update

When a track is lost for several frames due to occlusion, the Kalman filter keeps extrapolating with a possibly wrong velocity, so errors accumulate. Following OC-SORT, when the track is re-associated, a virtual trajectory is generated by linear interpolation between the last observation before occlusion and the new observation, and the Kalman filter is re-updated along this virtual trajectory. This corrects the velocity estimate and reduces drift after occlusion.

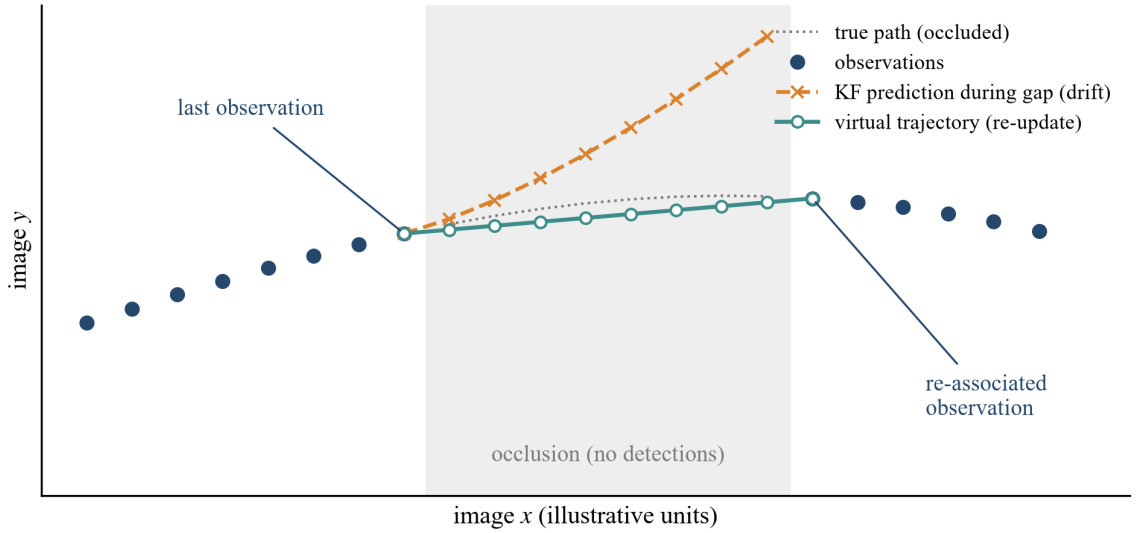


Figure 3.3 Occlusion handling: during a detection gap the Kalman prediction drifts, and on re-association OC-SORT re-updates the filter along a virtual trajectory between the last and the new observation (illustrative, synthetic data).

Error accumulation. Suppose a track is last observed at time t_1 and its velocity estimate contains an error δv. During T frames without observations, the position error grows linearly as $T\,\delta v$, while the covariance grows as $P_{t1+T} = F^T P_{t1} (F^T)^T + \Sigma_k F^k Q (F^k)^T$, whose velocity-coupled position terms grow with T^2 to T^3. When the target reappears at time $t_2 = t_1 + T$ and is matched, a single standard update moves the state towards the new observation, but the velocity is only partially corrected and the filter has no memory of the true path taken during the occlusion; the corrupted velocity then causes further mismatches in subsequent frames.

Re-update. Let z_{t1} be the last real observation and z_{t2} the new one. A virtual observation sequence is generated for the occluded interval:

$$\tilde{z}_t = z_{t1} + ((t - t_1) / (t_2 - t_1)) (z_{t2} - z_{t1}),\ t_1 < t \leq t_2 \quad (3.10)$$

The filter state is rolled back to the posterior saved at t_1, ($\hat{x}_{t1}$, P_{t1}), and the predict–update cycle is run again for $t = t_1 + 1, ..., t_2$ using $\tilde{z}_t$ as measurements. Because consecutive virtual observations differ by the constant step $(z_{t2} - z_{t1}) / T$, the velocity component converges towards this average displacement per frame, which is the best constant-velocity explanation of the two real observations. The re-updated covariance is also reduced to a level consistent with continuous observation, so the gate does not remain unnecessarily wide after recovery. The cost of re-update is T extra Kalman steps for the recovered track only, which is negligible.

3.4.4 Motion-direction consistency

For each track, the direction of motion is estimated from observations Δt frames apart. For a candidate detection, the angle between the track's direction and the direction from the track's last observation to the detection is computed. A large angle indicates an implausible association.

$$c_{dir}(i, j) = | \theta_{track,i} - \theta_{i \to j} | / \pi \quad (3.11)$$

The track direction is computed from real observations rather than from the Kalman velocity, because the latter is affected by drift during occlusion:

$$\theta_{track,i} = atan2(y_{i,t} - y_{i,t-\Delta t}, x_{i,t} - x_{i,t-\Delta t}),\ \theta_{i \to j} = atan2(y_j - y_{i,t}, x_j - x_{i,t}) \quad (3.12)$$

where $(x_{i,t}, y_{i,t})$ is the centre of the last observation and $\Delta t = 3$ frames. If no observation exists exactly Δt frames earlier, the nearest earlier observation is used. The angular difference is wrapped to $[0, \pi]$ before normalisation. Two safeguards are applied: when the track's displacement over Δt is smaller than 0.1 of the box height (an almost stationary person, for whom the direction is undefined), and when the detection is closer to the last observation than this same threshold, c_{dir} is set to zero. This prevents the term from penalising standing pedestrians and small jitter, which are common in dwell areas.

3.5 Fused Association Cost

In the first association stage, the cost between track i and high-score detection j is:

$$C_{ij} = min(d_{IoU}(i, j), \beta\, d_{app}(i, j)) + \gamma\, c_{dir}(i, j) \quad (3.13)$$

where $d_{IoU} = 1 - IoU$ and d_{app} is the cosine distance of ReID embeddings. Taking the minimum of the motion and appearance distances (as in BoT-SORT) lets either cue establish a match when the other is unreliable. Gating rejects pairs whose IoU distance exceeds 0.5 *and* whose appearance distance exceeds 0.25, preventing matches that are inconsistent in both cues. The weights β and γ are set on the validation split.

3.5.1 Gating logic in detail

Before assignment, the raw distances are processed by a sequence of gates. Each gate sets the corresponding entry to a large constant C_∞ (for example 10^5), which the assignment solver will never choose; after solving, any pair whose cost exceeds the acceptance threshold $\tau_1 = 0.8$ is rejected.

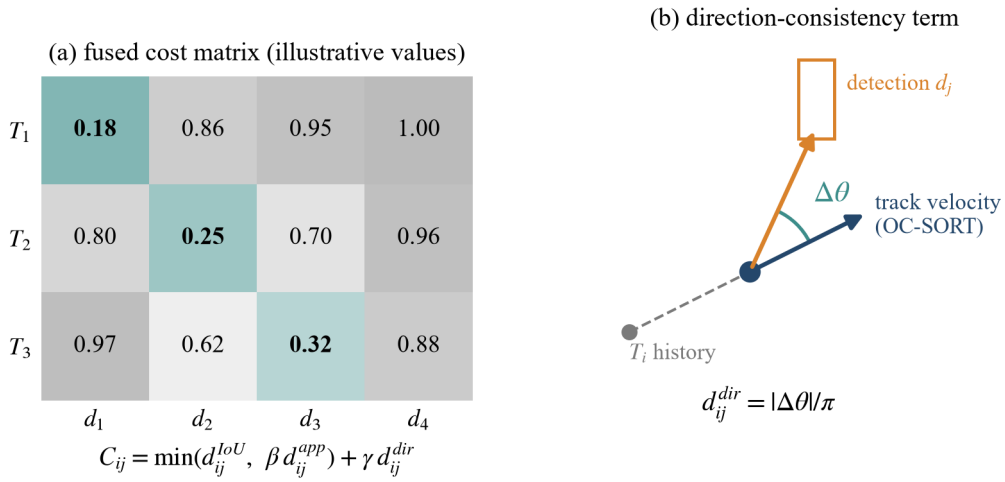


Figure 3.4 (a) Example fused cost matrix combining IoU, appearance and direction terms (illustrative values); (b) direction-consistency angle between track velocity and the track-to-detection vector.

- **Appearance gate.** If $d_{app}(i, j) > \theta_{app} = 0.25$, the appearance distance is set to 1 (uninformative), so that appearance cannot contribute to the minimum.
- **Proximity gate.** If $d_{IoU}(i, j) > \theta_{IoU} = 0.5$, the appearance distance is also set to 1. This prevents a visually similar person far away from being matched on appearance alone, a typical failure in crowds where many people wear similar clothing.
- **Motion gate.** If the Mahalanobis distance exceeds the χ^2 threshold for a confirmed track that was observed in the previous frame, the whole entry is set to C_∞. For lost tracks the motion gate is relaxed by a factor equal to the number of lost frames (capped at 5), reflecting the larger uncertainty.
- **Missing appearance.** When ReID is skipped by the adaptive trigger (see below), d_{app} is undefined and the cost reduces to $d_{IoU} + \gamma\, c_{dir}$.

With these gates, the effective cost is

$$C_{ij} = min(\, d_{IoU},\, \beta\, \tilde{d}_{app}\,) + \gamma\, c_{dir},\ \tilde{d}_{app} = d_{app}\ if\ d_{app} \le \theta_{app}\ and\ d_{IoU} \le \theta_{IoU},\ otherwise\ 1 \tag{3.14}$$

The weight $\beta \le 1$ makes appearance more influential when β is small. Since $\tilde{d}_{app}$ is at most θ_{app} when it is informative, choosing $\beta = 1$ means that appearance wins the minimum only for pairs with good visual similarity and moderate overlap, and the direction term with weight γ then breaks ties among candidates that are equally plausible

by position and appearance. The sensitivity of the results to β and γ is studied in Chapter 6.

3.5.2 Track appearance model

Each track keeps an exponentially smoothed embedding. To avoid polluting it with features from occluded crops, the update weight depends on the detection confidence s, as proposed in Deep OC-SORT:

$$\alpha_t = \alpha_f + (1 - \alpha_f)\,(1 - (s - \sigma)/(1 - \sigma)),\ f_{track} \leftarrow \alpha_t f_{track} + (1 - \alpha_t) f_{det} \qquad (3.15)$$

with $\alpha_f = 0.95$ and $\sigma = 0.6$, the high-score threshold. A confident detection (s close to 1) updates the template with weight $1 - \alpha_f$, while a detection at the threshold leaves it unchanged. The template is re-normalised to unit length after each update. In addition, the embedding is not updated when the detection box overlaps another detection with IoU above 0.3, because such crops contain parts of two people.

3.5.3 Adaptive ReID trigger

To keep the tracker efficient, appearance features are extracted only for high-score detections, with a lightweight OSNet-x0.25 model in batched mode, and only when the scene is crowded — specifically when the predicted box of a track overlaps with another track (IoU > 0.3). In sparse scenes, IoU alone is sufficient and the ReID step is skipped. This adaptive strategy reduces the average ReID cost substantially in typical surveillance scenes.

The decision is taken per detection rather than per frame, so that a single crowded group in an otherwise empty scene does not trigger feature extraction for everybody. The procedure is as follows.

- **Step 1.** After Kalman prediction and CMC, compute the pairwise IoU matrix among all predicted boxes of confirmed and lost tracks, and mark track i as *ambiguous* if $\max_{k \neq i} \mathrm{IoU}(\hat{b}_i, \hat{b}_k) > \theta_{occ} = 0.3$ or if track i has been lost for at least one frame.
- **Step 2.** For each high-score detection j, compute its IoU with all predicted boxes. Mark j as *needing appearance* if it overlaps (IoU > 0.1) any ambiguous track, if it overlaps two or more tracks with IoU > 0.3, or if it overlaps no track at all (a potential re-appearance of a lost track).
- **Step 3.** Every $N_{ref} = 10$ frames, additionally mark all high-score detections that match a confirmed track with IoU > 0.7, so that track templates are refreshed periodically even in sparse scenes; this keeps templates current for later re-linking.
- **Step 4.** Crop the marked detections, resize them to 256 × 128, and run OSNet once on the batch; store the embeddings with the detections. Unmarked detections receive

no embedding.

- **Step 5.** Build the fused cost matrix. For pairs where either the track template or the detection embedding is missing, use the appearance-free cost.

The fraction of detections passed to the ReID network, ρ, is logged per frame; the expected ReID cost per frame is then ρ times that of always-on ReID plus the fixed overhead of the IoU computation, which is negligible. In the worst case (a dense crowd), ρ approaches 1 and the cost equals that of always-on ReID, so the adaptive strategy never performs worse than the non-adaptive one.

3.6 Two-Stage Association and Track Management

- **Stage 1**: high-score detections ($s > 0.6$) are associated with all tracks (confirmed and lost) using the fused cost.
- **Stage 2**: remaining tracks are associated with low-score detections ($0.1 < s \leq 0.6$) using IoU only, because appearance features of occluded, low-score boxes are unreliable.
- **Unconfirmed tracks**: tracks created in the previous frame are associated with remaining high-score detections; a track becomes confirmed after it is matched in 2 consecutive frames, suppressing short false-positive tracks.
- **Birth**: unmatched high-score detections with $s > 0.7$ start new tracks.
- **Death**: tracks unmatched for more than 30 frames (adjusted to the frame rate) are removed from the active set but stored for post-processing.

3.7 Algorithm Pseudocode

- Input: video frames $I_1 ... I_T$; detector; ReID model.
- Output: trajectories $\{\tau_k\}$.
- For each frame t: $D \leftarrow \text{Detect}(I_t)$; split D into D_{high}, D_{low}; $A_t \leftarrow \text{CMC}(I_{t-1}, I_t)$; for each track: predict with Kalman, warp by A_t.
- Compute fused cost between tracks and D_{high}; Hungarian matching; update matched tracks (with OC re-update for tracks recovering from occlusion).
- Compute IoU cost between remaining tracks and D_{low}; Hungarian matching; update.
- Handle unconfirmed tracks, births and deaths.
- After the sequence: apply trajectory post-processing (Chapter 4).

3.8 Chapter Summary

This chapter presented PMV-Track, which combines a viewpoint-adapted detector, camera motion compensation, observation-centric Kalman re-update, a fused IoU–appearance–direction association cost with adaptive ReID, and two-stage association.

Chapter 4
Trajectory Post-Processing and Motion Visualization

4.1 Trajectory Post-Processing

Visualization depends on complete and smooth trajectories, so offline or windowed post-processing is applied. In offline mode the whole sequence is processed after tracking; in online mode, post-processing runs on a sliding window of W = 10 s that lags the live stream by G_{max}, so that a tracklet is finalised only when no later tracklet can be linked to it. The three steps — re-linking, interpolation and smoothing — are applied in this order, because interpolation needs the gaps created by linking and smoothing needs the complete sequence.

4.1.1 Tracklet re-linking

Long occlusions can still break one pedestrian's trajectory into several tracklets. A tracklet τ_a that ends at time t_a^{end} may be linked to a tracklet τ_b that starts at t_b^{start} if $0 < t_b^{start} - t_a^{end} \leq G_{max}$ (e.g., 2 seconds), the spatial gap is consistent with walking speed (at most v_{max} = 2.5 m/s on the ground plane), and the mean appearance distance is small. Candidate links are scored as:

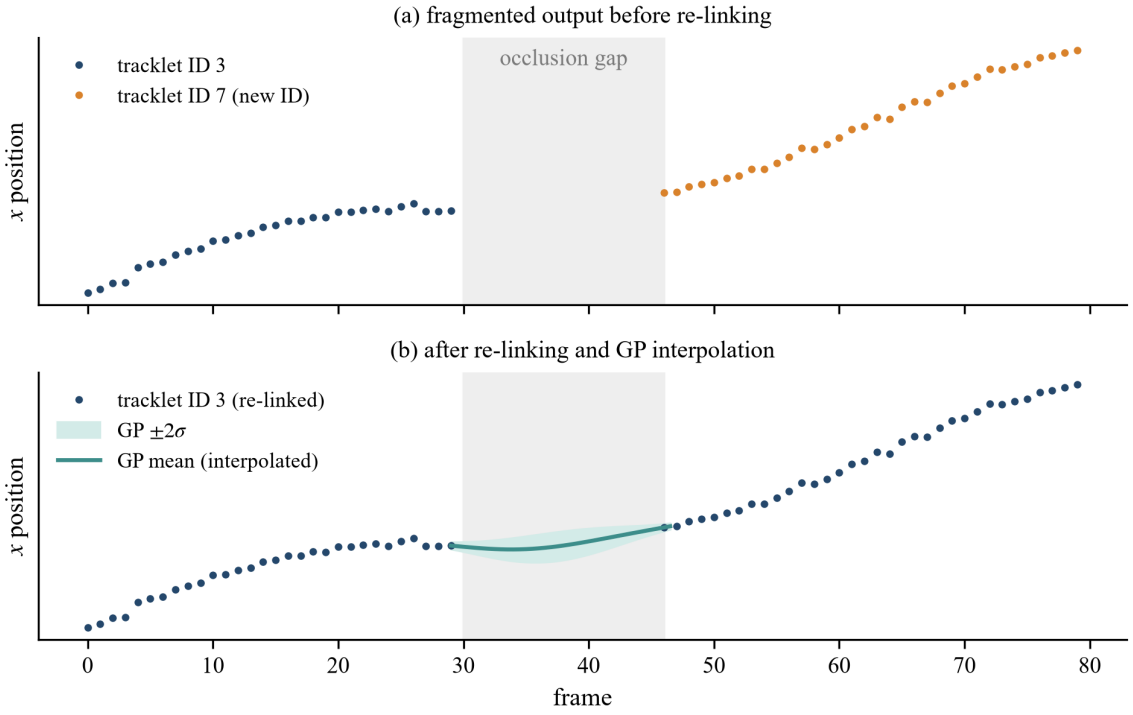


Figure 4.1 Tracklets fragmented by occlusion (a) are re-linked into one identity and the gap is filled by Gaussian-process interpolation with its ±2σ band (b) (illustrative, synthetic data).

$$S(a, b) = w_1 d_{app}(a, b) + w_2 d_{pos}(a, b) / (v_{max} \Delta t) + w_3 \Delta t / G_{max} \quad (4.1)$$

and the global linking is solved as a minimum-cost assignment, with a threshold to reject unlikely links.

Tracklet descriptors. For each tracklet, a robust appearance descriptor is computed as the normalised mean of the embeddings of its $K_{app} = 10$ highest-confidence, non-overlapping detections, separately for its first and last seconds (f_a^{end}, f_b^{start}), since clothing appearance may change slightly with viewpoint along a long path. Then $d_{app}(a, b) = 1 - f_a^{end} \cdot f_b^{start}$. If a tracklet has no stored embeddings, d_{app} is set to 0.5, a neutral value.

Motion-predicted position gap. Rather than comparing the raw end and start positions, the end of τ_a is extrapolated forward and the start of τ_b backward with their ground-plane velocities estimated over their last and first second (v_a, v_b), and the gap is measured at the midpoint of the interval:

$$d_{pos}(a, b) = \| (P_a^{end} + v_a \Delta t/2) - (P_b^{start} - v_b \Delta t/2) \|, \Delta t = t_b^{start} - t_a^{end} \quad (4.2)$$

The extrapolation is clipped to a speed of v_{max} to prevent unreliable velocity estimates from very short tracklets from dominating.

Hard constraints. A pair (a, b) is admissible only if (i) $0 < \Delta t \leq G_{max}$; (ii) the straight-line distance $\|P_b^{start} - P_a^{end}\| \leq v_{max} \Delta t + r_0$, where $r_0 = 0.5$ m absorbs localisation error; (iii) the two tracklets do not overlap in time; and (iv) $d_{app}(a, b) \leq 0.4$. Inadmissible pairs have infinite cost.

Assignment. Let E be the set of tracklets that end before the end of the window and B the set that start after its beginning. A cost matrix of size $|E| \times |B|$ is formed with entries $S(a, b)$ for admissible pairs and C_∞ otherwise, and solved with LAPJV [29]. Assigned pairs with $S(a, b) > \tau_{link} = 0.6$ are rejected. Each accepted link merges τ_b into τ_a, keeping the identity of τ_a. Because a single round of one-to-one assignment can only join pairs, the procedure is repeated until no further link is accepted (in practice two to three rounds), which allows chains of three or more tracklets to be joined. The weights are set to $w_1 = 0.5$, $w_2 = 0.3$, $w_3 = 0.2$ and tuned on the validation split.

4.1.2 Gap interpolation

Missing frames inside a trajectory are filled. Linear interpolation is used for short gaps; for longer gaps, Gaussian process regression (as in the GSI method of StrongSORT) is used, giving smooth, plausible paths.

For gaps of at most $g_{lin} = 5$ frames, each box coordinate is interpolated linearly between the two bounding observations. For longer gaps (up to the re-linking limit G_{max}), each coordinate $c(t) \in \{x_c, y_c, w, h\}$ is modelled independently as a Gaussian process [56]

with zero mean (after subtracting a linear trend) and a squared-exponential kernel:

$$k(t, t') = \sigma_f^2 \exp(-(t - t')^2 / (2 \ell^2)) + \sigma_n^2 \delta(t, t') \quad (4.3)$$

Given the n observed time stamps $t = [t_1, ..., t_n]$ in a context window of ±2 s around the gap, with observations c, the posterior mean and variance at a missing time t_* are

$$\mu(t_*) = k_*^T (K + \sigma_n^2 I)^{-1} c, \sigma^2(t_*) = k(t_*, t_*) - k_*^T (K + \sigma_n^2 I)^{-1} k_* \quad (4.4)$$

where $K_{pq} = k(t_p, t_q)$ and $k_* = [k(t_*, t_1), ..., k(t_*, t_n)]^T$. Following GSI, the length scale is made adaptive to the tracklet length, $\ell = \tau \log(\tau^3 / L)$, clipped to a valid range, where L is the number of observations and τ a constant (τ = 10); longer tracklets therefore receive smaller length scales and can follow more detail. The posterior mean replaces both the missing and the observed values inside the window, which also smooths the observed boxes. The posterior variance is stored and used to render interpolated segments with a dashed line, so that the viewer can distinguish observed from inferred motion.

4.1.3 Smoothing

Box jitter causes noisy foot-point positions and inflated speed estimates. Trajectories in ground-plane coordinates are smoothed with a Savitzky–Golay filter [25] (or a Rauch–Tung–Striebel Kalman smoother [57]), which preserves turning points better than a moving average.

The Savitzky–Golay filter fits, in a sliding window of 2m + 1 samples centred on sample n, a polynomial of degree d by least squares and replaces the centre value by the polynomial evaluated at the centre. Because the fit is linear in the data, it is equivalent to a convolution with fixed coefficients:

$$\tilde{p}_n = \Sigma_{k=-m}^{m} c_k p_{n+k}, c = e_0^T (J^T J)^{-1} J^T \quad (4.5)$$

where J is the (2m + 1) × (d + 1) Vandermonde matrix with entries $J_{kq} = k^q$, k = −m, ..., m, and e_0 selects the constant term. The first derivative is obtained by selecting the linear term instead, giving a smoothed velocity estimate directly. A moving average corresponds to d = 0 and flattens turns and peaks of speed; with d = 2 or 3, quadratic and cubic segments of the path are reproduced exactly, so turns are preserved while high-frequency jitter is attenuated. The window length is specified in seconds (1 s, i.e. m = 12 at 25 FPS) so that the smoothing has the same physical meaning across frame rates, and d = 2. At trajectory ends, where the symmetric window is not available, the polynomial fit of the nearest full window is evaluated at the end positions.

The RTS smoother is an alternative that runs a forward Kalman filter on ground-plane positions followed by a backward pass:

$$\hat{x}_{k|N} = \hat{x}_{k|k} + G_k \left(\hat{x}_{k+1|N} - \hat{x}_{k+1|k} \right),\ G_k = P_{k|k} F^T P_{k+1|k}^{-1} \tag{4.6}$$

It is optimal under the linear-Gaussian model but tends to over-smooth sharp turns, which are frequent at corridor junctions; both options are compared in Chapter 6.

4.2 Ground-Plane Projection

The homography H is estimated once per camera from four or more manually selected correspondences between image points and measured ground coordinates (in metres). For each box, the foot point $u_f = ((x_1 + x_2)/2, y_2)$ is projected to the ground plane. When the bottom of a box is occluded, the foot point is estimated from the box top and the average height-to-position relation of that camera. All motion statistics are computed in ground coordinates, which removes perspective distortion.

4.2.1 Homography estimation by normalised DLT

Given $n \geq 4$ correspondences $(u_i, v_i) \leftrightarrow (X_i, Y_i)$, the relation $[X_i, Y_i, 1]^T \propto H\,[u_i, v_i, 1]^T$ is rewritten as a cross product equal to zero, which yields two linearly independent equations per correspondence in the nine entries $h = [h_1, ..., h_9]^T$ of H (row-major):

$$[\, -u_i \; -v_i \; -1 \; 0 \; 0 \; 0 \; X_i u_i \; X_i v_i \; X_i \,]\, h = 0 \tag{4.7}$$

$$[\, 0 \; 0 \; 0 \; -u_i \; -v_i \; -1 \; Y_i u_i \; Y_i v_i \; Y_i \,]\, h = 0 \tag{4.8}$$

Stacking all equations gives a $2n \times 9$ system $A\,h = 0$. Subject to $\|h\| = 1$, the least-squares solution is the right singular vector of A associated with the smallest singular value [21].

The DLT is not invariant to the choice of coordinates, and image coordinates in pixels (of order 10^3) mixed with ground coordinates in metres make A badly conditioned. Following Hartley [60], both point sets are normalised before estimation: each set is translated so that its centroid is at the origin and scaled isotropically so that the average distance from the origin is $\sqrt{2}$. With normalising similarity transforms T_{img} and T_{gnd}, the DLT is applied to the normalised points to obtain $\tilde{H}$, and the final homography is de-normalised:

$$H = T_{gnd}^{-1} \tilde{H} T_{img} \tag{4.9}$$

and scaled so that $h_9 = 1$. When more than four correspondences are available, the DLT estimate initialises a Levenberg–Marquardt refinement that minimises the geometric error in ground coordinates. If the calibration points may contain gross mistakes (for example a mis-clicked point), RANSAC over minimal four-point samples is used first.

4.2.2 Reprojection error

The quality of the calibration is reported as the root-mean-square transfer error over the calibration points, and — more importantly — over a separate set of held-out check

points measured on the ground but not used for estimation:

$$e_{RMS} = sqrt((1/n) \Sigma_i || (X_i, Y_i) - \pi(H [u_i, v_i, 1]^T) ||^2) \quad (4.10)$$

where $\pi([a, b, c]^T) = (a/c, b/c)$ is the dehomogenisation. The error is expressed in metres. The calibration tool also renders a projected metric grid on the image so that gross errors are visible immediately. Because the projection is non-linear, a constant pixel error produces a ground error that grows with distance from the camera; the tool therefore reports the error separately for near, middle and far thirds of the image, and the analysis region can be limited to the area where the expected error is below a chosen tolerance (e.g., 0.3 m).

4.2.3 Foot-point estimation under occlusion

When the lower body is occluded (by another person, a barrier or the image border), the bottom edge of the box does not lie on the ground, and projecting it would place the person too close to the camera. Occlusion of the foot point is detected when (i) the box bottom touches the image border, (ii) the box overlaps another box whose bottom edge is lower in the image by more than 5 % of the box height, or (iii) the box aspect ratio h/w falls below 0.6 of the running median of the track, indicating truncation.

In these cases, the foot point is estimated from the head position. For a camera without strong roll looking at a planar ground, the image height h(v) of a standing person whose foot is at image row v is approximately linear in v. A robust linear regression $h = a\, v_f + b$ is fitted over all unoccluded, high-confidence detections in the calibration sequence. Given the top edge y_1 of an occluded box, the foot row v_f solves $v_f - y_1 = a\, v_f + b$, i.e.

$$v_f = (y_1 + b) / (1 - a), \; u_f = (x_1 + x_2) / 2 \quad (4.11)$$

If the track was recently unoccluded, a track-specific estimate based on its last unoccluded height, rescaled by the regression, is preferred, because individual heights vary. Estimated foot points are flagged in the output data, and the Kalman measurement noise of the ground-plane smoother is increased for them.

4.3 Motion Features

For each trajectory, the following features are computed from smoothed ground-plane positions p_t with frame interval Δ:

$$v_t = || p_{t+1} - p_{t-1} || / (2\Delta), \; \varphi_t = atan2(Y_{t+1} - Y_{t-1}, X_{t+1} - X_{t-1}) \quad (4.12)$$

- Instantaneous speed v_t (m/s) and heading φ_t.
- Path length, mean speed and straightness (net displacement / path length).
- Stop events, when $v_t < 0.2$ m/s for at least 2 s, and dwell time per region.

- Entry and exit points, giving origin–destination pairs.

Path length and straightness are defined as

$$L = \Sigma_t \| p_{t+1} - p_t \|, \sigma_{str} = \| p_T - p_1 \| / L \quad (4.13)$$

with σ_{str} close to 1 for direct walks and small values for wandering or loitering. To avoid detecting false stops from single noisy frames, the stop detector uses hysteresis: a stop begins when the speed drops below 0.2 m/s and ends only when it exceeds 0.4 m/s for at least 0.5 s. Entry and exit points are the first and last positions of a re-linked trajectory, and are assigned to user-defined zones by point-in-polygon tests.

4.4 Visualization Methods

4.4.1 Trajectory overlay with fading tails

Each active pedestrian is drawn with its box, ID and a polyline of its last K positions whose opacity fades with age. Colours are assigned per ID from a perceptually distinct palette. This view supports real-time monitoring and qualitative evaluation of the tracker (Figure 4.1).

For a tail point of age a frames ($0 \le a \le K$), the opacity is $\alpha(a) = \alpha_0 (1 - a/K)^{\gamma f}$ with $\gamma_f = 1.5$, so recent motion is emphasised. The colour of an ID is chosen by the golden-angle rule on the hue circle, hue(id) = (id × 137.5°) mod 360°, at fixed saturation and lightness, which keeps consecutive IDs visually distinct.

[Figure 4.2 placeholder — insert diagram or result image here]

Figure 4.2 Trajectory overlay with fading tails and per-ID colours.

4.4.2 Speed- and direction-coloured trajectories

Accumulated trajectories over a time window are drawn on a background frame (or a bird's-eye map), with colour encoding speed (sequential colour map) or heading (cyclic colour map). This reveals fast corridors, slow zones and the main directions of flow. The speed colour map is perceptually uniform (e.g., viridis) and its range is fixed to [0, 2.5] m/s for all videos so that colours are comparable; heading uses a cyclic map so that 0 and 2π share a colour. Each segment is coloured by the smoothed value at its start, and segments are drawn with low opacity so that overlapping paths accumulate visually.

4.4.3 Occupancy and density heatmap

Kernel density estimation over all ground-plane positions in a time window gives an occupancy heatmap showing where pedestrians spend the most time. A per-frame density map (persons per m^2) is also computed; cells above a safety threshold (e.g., 4 persons per m^2) are highlighted as congestion warnings (Figure 4.2).

Occupancy. Each sample of each trajectory contributes one point per frame, so the occupancy map integrates person-seconds. With N samples p_i and a 2-D isotropic Gaussian kernel, the estimate in the kernel density estimator of Chapter 2 is evaluated on a ground grid of 0.1 m cells and multiplied by $N\Delta$ to obtain units of person-seconds per m^2. The evaluation is performed efficiently by accumulating counts in a 2-D histogram and convolving it with a discrete Gaussian, which is exact up to binning.

Bandwidth. Silverman's rule of thumb [55] for d-dimensional data gives

$$h = \hat{\sigma}\,(4 / ((d + 2)\, N))^{1/(d+4)}, \text{ which for } d = 2 \text{ is } h = \hat{\sigma} N^{-1/6} \tag{4.14}$$

where $\hat{\sigma}$ is the average marginal standard deviation. Because positions in a scene are strongly multi-modal (several corridors), the global standard deviation over-estimates the appropriate scale, and the rule produces over-smoothed maps. A physical bandwidth h = 0.5 m, comparable to the body-ellipse radius used in pedestrian planning, is therefore used by default, and the Silverman value is offered as an alternative.

Instantaneous density. For crowd safety, the instantaneous density at location x and time t is computed with a kernel normalised to integrate to one per person:

$$\rho(x, t) = \Sigma_{i \in A(t)} (1 / (2\pi\, h_\rho^{\,2})) \exp(-||x - p_{i,t}||^2 / (2\, h_\rho^{\,2})) \tag{4.15}$$

where A(t) is the set of pedestrians present at t and $h_\rho = 0.7$ m. Density is then mapped to Fruin's level of service (LoS) [26], which classifies walkways by the area available per person. Table 4.1 gives the walkway thresholds expressed as density. LoS E and F are highlighted as warnings.

LoS	Area per person (m^2)	Density (persons/m^2)	Description
A	≥ 3.25	≤ 0.31	Free circulation
B	2.32 – 3.25	0.31 – 0.43	Minor conflicts
C	1.39 – 2.32	0.43 – 0.72	Restricted speed, crossing difficult
D	0.93 – 1.39	0.72 – 1.08	Most pedestrians restricted
E	0.46 – 0.93	1.08 – 2.17	Severely restricted, shuffling

LoS	Area per person (m^2)	Density (persons/m^2)	Description
F	≤ 0.46	≥ 2.17	Breakdown, frequent contact

Table 4.1 Fruin levels of service for walkways, expressed as area per person and density.

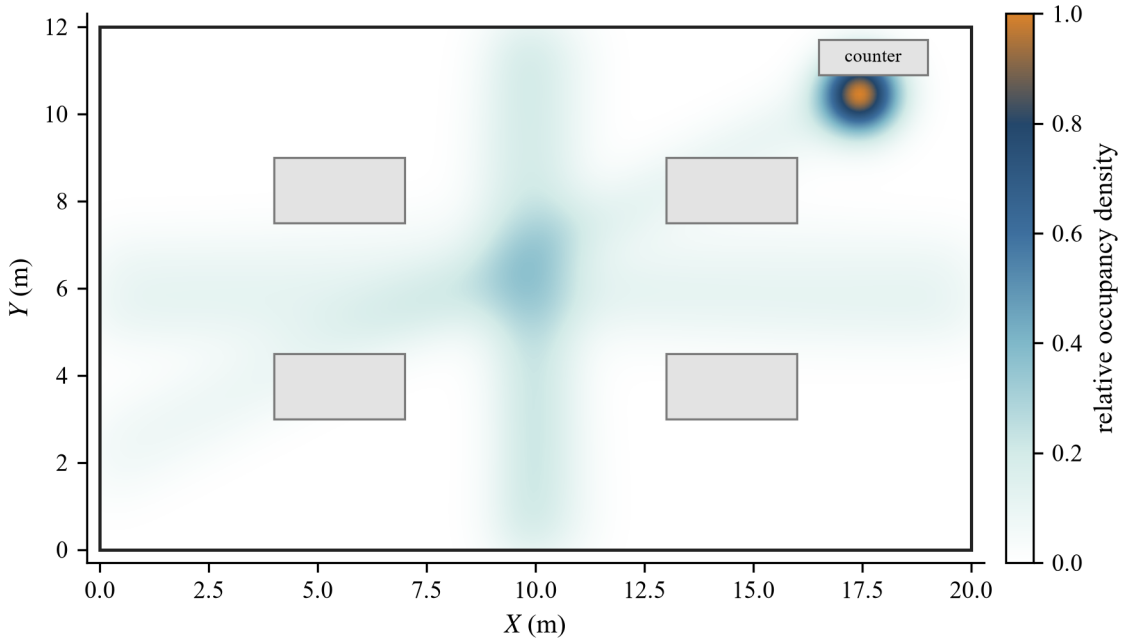


Figure 4.3 Kernel-density occupancy heatmap on a bird's-eye floor plan with obstacles and a counter (illustrative, synthetic data).

4.4.4 Flow field map

The ground plane is divided into a grid (e.g., 0.5 m cells). For each cell, the mean velocity vector of pedestrians passing through it is computed; arrows show direction and their length or colour show magnitude. Cells with low counts are omitted. Entropy of headings in a cell indicates whether flow is orderly (low entropy) or chaotic (high entropy).

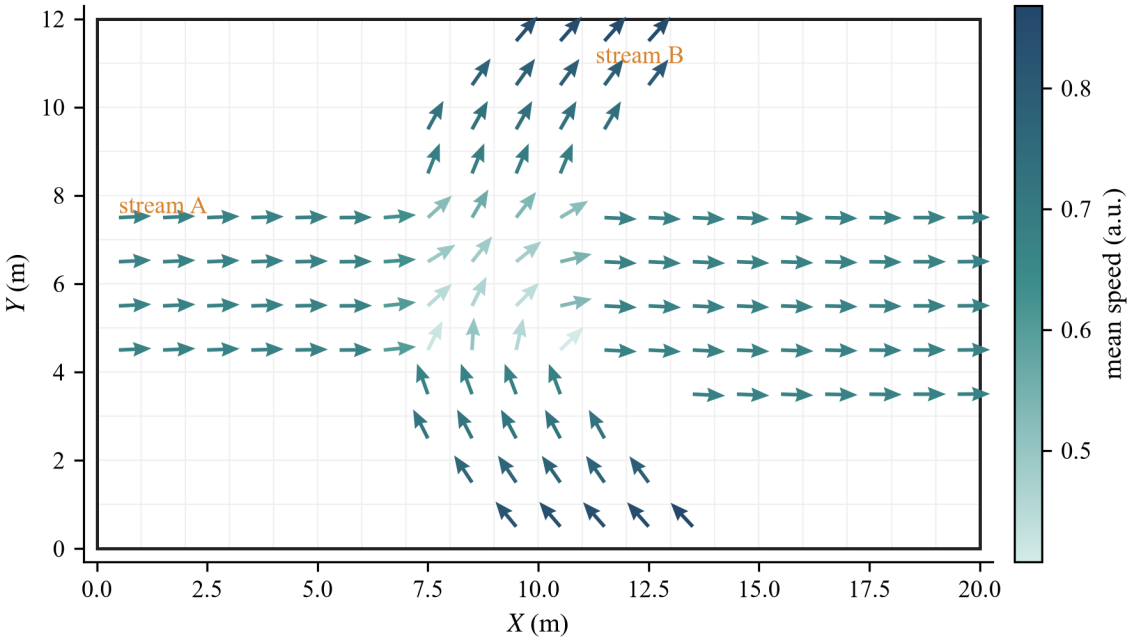


Figure 4.4 Grid flow field of mean motion direction per 1 m cell, showing two crossing pedestrian streams (illustrative, synthetic data).

Averaging. Let S_c be the set of trajectory samples whose position falls in cell c during the analysis window. To avoid long-dwelling pedestrians dominating a cell, each trajectory contributes the mean of its own velocities in the cell, and the cell average is taken over trajectories:

$$\bar{v}_c = (1 / |T_c|) \sum_{k \in Tc} \left((1 / |S_{c,k}|) \sum_{t \in Sc,k} v_{k,t} \right) \tag{4.16}$$

where T_c is the set of trajectories passing through c and $v_{k,t}$ the velocity vector. Cells with $|T_c|$ below $n_{min} = 5$ are omitted. Note that in counter-flow the mean vector can be close to zero although many people walk fast; the coherence ratio $\kappa_c = \|\bar{v}_c\| / \text{mean}_k \|\bar{v}_{c,k}\| \in [0, 1]$ is therefore shown together with the arrow (arrow opacity proportional to κ_c).

Heading entropy. Headings in each cell are quantised into B = 8 sectors of 45°. With $p_{c,b}$ the fraction of samples (weighted per trajectory as above) in sector b, the normalised entropy is

$$E_c = - (1 / \log B) \sum_{b=1}^{B} p_{c,b} \log p_{c,b} \tag{4.17}$$

which is 0 when all pedestrians move in the same direction and 1 when headings are uniformly distributed. Bidirectional flow gives an intermediate value ($\log 2 / \log 8 = 1/3$), which helps distinguish lane-formed counter-flow from disordered motion.

4.4.5 Dwell-time map and origin–destination diagram

The dwell-time map shows the average time pedestrians stop in each cell. Entry and exit zones are defined, and an origin–destination matrix is visualised as a chord diagram or flow arrows between zones.

The dwell time of trajectory k in cell (or zone) c is the total duration of its stop events located in c, $D_{k,c} = \Delta \Sigma_t 1[p_{k,t} \in c]\, 1[\text{stop}_{k,t}]$. The map shows the mean over trajectories that stopped in c, $\bar{D}_c = (1/|K_c|)\, \Sigma_{k \in Kc}\, D_{k,c}$, together with the number of stopping trajectories $|K_c|$, so that a single long-staying person is not confused with a popular waiting area. The origin–destination matrix has entries OD_{rs} equal to the number of trajectories that enter in zone r and exit in zone s; trajectories that begin or end in the interior of the scene (for example because tracking is lost) are counted in a separate "unknown" row or column, which makes residual fragmentation visible.

4.4.6 Virtual-line counting and statistics

The user draws one or more virtual lines. A crossing is counted when a trajectory's ground-plane segment intersects the line, with direction given by the sign of the cross product. Using trajectories after re-linking prevents double counting. Counts are displayed per time interval as a bar chart, together with real-time occupancy.

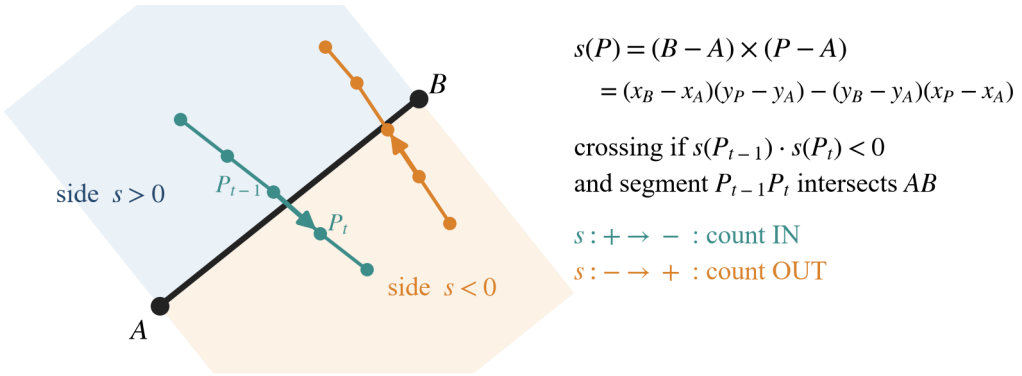


Figure 4.5 Virtual-line crossing test: the sign of the cross product s(P) changes when a trajectory crosses line AB, and the sign transition gives the counting direction.

Line-crossing test. Let the virtual line be the segment from A to B and the trajectory segment from P = p_t to P' = p_{t+1}. With the 2-D cross product $(a \times b) = a_x b_y - a_y b_x$, define

$$s_1 = (B - A) \times (P - A),\ s_2 = (B - A) \times (P' - A),\ s_3 = (P' - P) \times (A - P),\ s_4 = (P' - P) \times (B - P) \quad (4.18)$$

The segments intersect if $s_1 s_2 < 0$ and $s_3 s_4 < 0$. The direction of crossing is given by the sign of s_2: positive means a crossing from the right to the left of the directed line A→B. Jitter of a person standing on the line could generate repeated crossings in alternating directions; this is prevented by a hysteresis band of half-width b = 0.3 m: a crossing is registered only when the trajectory moves from one side of the band to the other, i.e.

when the signed distance s / ||B − A|| changes from below −b to above +b (or vice versa). Each trajectory may cross a given line at most once per direction within a refractory period of 2 s.

Counts are aggregated in user-selected intervals (e.g., 1 min), and the occupancy of each zone is computed as the number of trajectories inside it at each frame, smoothed over 1 s.

4.5 Chapter Summary

This chapter described trajectory re-linking, interpolation and smoothing, ground-plane projection, motion feature extraction, and six visualization methods that turn tracking results into interpretable images and charts.

Chapter 5
System Design and Implementation

5.1 Requirements

- **Functional**: input from video files or RTSP streams; online tracking with live overlay; offline analysis with post-processing; camera calibration tool for homography; generation and export of visualizations (PNG, MP4) and data (CSV/JSON trajectories); configurable virtual lines and zones.
- **Non-functional**: real-time performance (at least 25 FPS at 1080p on a desktop GPU); modularity so that the detector and ReID models can be replaced; privacy — the option to blur faces in exported video and store only trajectories.

The functional requirements can be grouped into use cases for two user roles. An *analyst* loads a recorded video, calibrates the camera, defines lines and zones, runs offline analysis and exports figures and statistics. An *operator* connects to a live stream, watches the overlay and density warnings, and acknowledges alerts. Both roles share the calibration and configuration functions, which are stored per camera so that they need to be performed only once.

ID	Requirement	Priority	Verification
F1	Read MP4/AVI files and RTSP streams	High	Functional test
F2	Track pedestrians online and draw overlays	High	Live demonstration, FPS log
F3	Post-process trajectories offline or in a sliding window	High	Comparison of raw and processed tracks
F4	Calibrate a homography from clicked points	High	Reprojection error report
F5	Generate heatmaps, flow fields, dwell maps, OD matrices	High	Visual inspection, unit tests
F6	Count line crossings per interval	High	Manual counting comparison

ID	Requirement	Priority	Verification
F7	Export trajectories (CSV, JSON, MOT format) and figures	Medium	File validation
F8	Raise density alerts above a LoS threshold	Medium	Scripted scenario
N1	At least 25 FPS at 1080p on a desktop GPU	High	Runtime benchmark
N2	Replaceable detector and ReID models	Medium	Plug-in test with alternative model
N3	Face blurring and trajectory-only storage	Medium	Inspection of exported data

Table 5.1 Functional (F) and non-functional (N) requirements.

5.2 Architecture

The system has four layers (Figure 5.1): an **input layer** (video decoding with hardware acceleration), a **perception layer** (detector, ReID, PMV-Track), an **analysis layer** (post-processing, projection, features, statistics) and a **presentation layer** (overlay rendering, heatmaps, charts and a graphical user interface). Decoding, inference and rendering run in separate threads connected by bounded queues, so that the GPU is kept busy while frames are decoded and drawn.

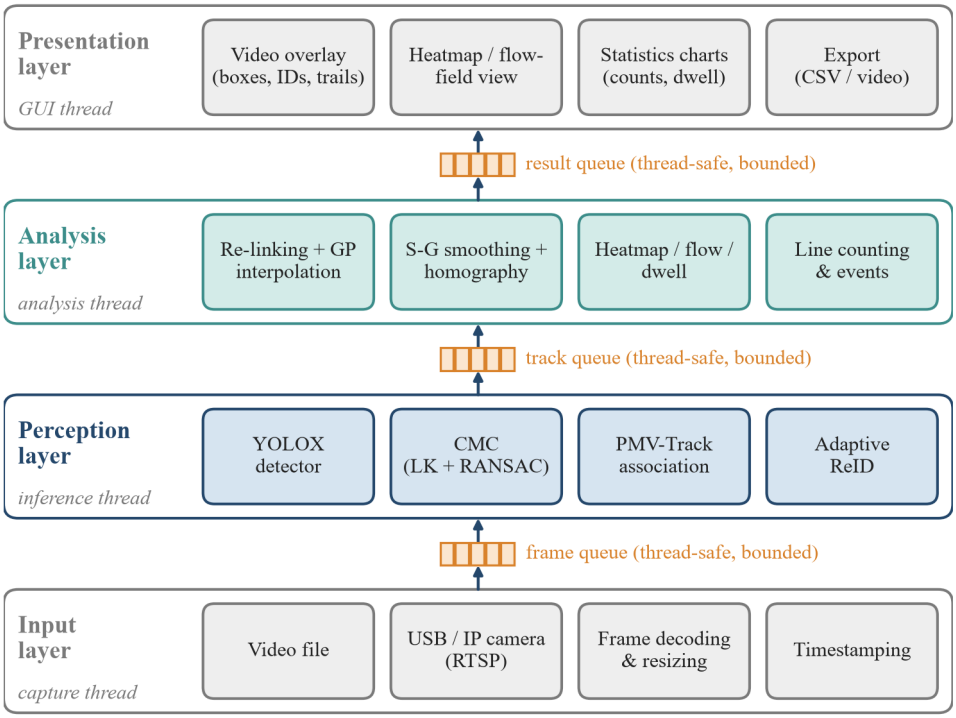


Figure 5.1 Four-layer system architecture (input, perception, analysis, presentation), each running in its own thread and connected by bounded thread-safe queues.

5.2.1 Modules

- **VideoSource**: wraps FFmpeg/OpenCV decoding of files and RTSP streams, provides frames with timestamps, and handles reconnection after network interruptions. For files it can seek and decode in a background thread; for streams it drops the oldest frame when the queue is full, so that latency stays bounded.
- **Detector**: an abstract interface with a method that takes a batch of frames and returns boxes and scores; the YOLOX implementation runs a TensorRT FP16 engine with letterbox pre-processing on the GPU.
- **ReIDExtractor**: crops and resizes the boxes selected by the adaptive trigger, runs OSNet-x0.25 in a single batch and returns normalised embeddings.
- **CMC**: implements the sparse optical-flow affine estimation of Chapter 3; it can be disabled per camera.
- **Tracker**: implements PMV-Track (Kalman filter, gating, fused cost, two-stage assignment, track life cycle) and emits per-frame track states.
- **PostProcessor**: buffers finished tracklets and applies re-linking, interpolation and smoothing in a sliding window or over the whole video.
- **Calibrator**: stores correspondences, estimates the homography and the height regression for foot-point estimation, and reports reprojection errors.

- **Analytics**: computes motion features, stops, zone occupancy, line crossings and OD matrices, and maintains the incremental grids for heatmaps and flow fields.
- **Renderer**: draws overlays on frames and renders bird's-eye views and charts.
- **Store**: writes trajectories and events to SQLite and exports CSV, JSON and MOTChallenge files.

5.2.2 Threading and data flow

The online pipeline uses four threads and three bounded queues. The *decode* thread pushes frames into Q_1 (capacity 8). The *inference* thread takes frames from Q_1, runs detection, CMC, ReID and tracking, and pushes (frame, tracks) pairs into Q_2 (capacity 8). The *analysis* thread consumes Q_2, projects tracks to the ground, updates the incremental statistics and the post-processing window, and pushes render jobs into Q_3. The *render/UI* thread draws and displays results. Because Python's global interpreter lock limits parallel CPU execution, the heavy work in each thread is performed by native libraries (CUDA, OpenCV, NumPy) that release the lock; if profiling shows contention, the analysis stage can be moved to a separate process with shared-memory frame buffers. Bounded queues provide back-pressure: when rendering is slow, the inference thread blocks rather than accumulating unbounded memory, and in live mode the decode thread drops frames instead.

Tracking itself is inherently sequential, since frame t depends on the state after frame t − 1, so throughput is improved within the inference thread by overlapping work: detection of frame t+1 is launched asynchronously on a CUDA stream while CMC and association of frame t run on the CPU.

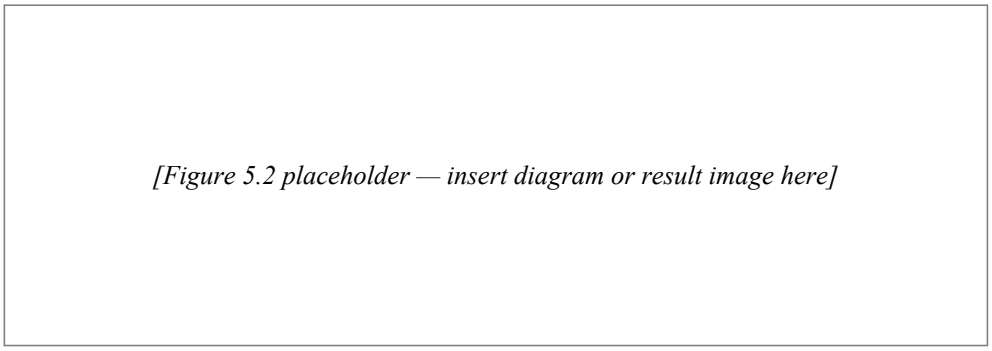


Figure 5.2 Thread and queue structure of the online pipeline.

5.2.3 Incremental statistics

Heatmaps and flow fields are maintained incrementally so that their cost does not grow with video length. The occupancy histogram, per-cell velocity sums, heading histograms and trajectory counts are stored as NumPy arrays updated in O(1) per sample; Gaussian convolution for display is performed only when the view is refreshed (once per second).

Because post-processing may later change a trajectory (by linking or smoothing), the incremental statistics are first computed from raw tracks for live display and then recomputed from final trajectories when the sliding window releases them; the offline report always uses final trajectories.

5.3 Implementation Environment

Component	Technology
Language	Python 3.x
Deep learning	PyTorch, TensorRT (FP16) for deployment
Detector / ReID	YOLOX (person), OSNet-x0.25
Video and drawing	OpenCV, FFmpeg
Visualization	Matplotlib, OpenCV colour maps
User interface	PyQt or a web dashboard (e.g. Streamlit)
Data storage	CSV / JSON trajectory files, SQLite

Table 5.2 Implementation technologies.

5.4 Data Format

Each trajectory record contains: frame index, timestamp, track ID, box (x_1, y_1, x_2, y_2), score, ground-plane position (X, Y), speed and heading. The MOTChallenge text format is also exported for evaluation.

5.4.1 Database schema

The SQLite store uses four tables. Indices on (camera_id, frame) and (track_id) make both per-frame queries (for replay) and per-trajectory queries (for analysis) efficient.

Table	Field	Type	Description
cameras	camera_id	INTEGER	Primary key
cameras	name, fps, width, height	TEXT, REAL, INTEGER	Source metadata
cameras	homography	TEXT (JSON)	3 × 3 matrix, row-major
cameras	height_model	TEXT (JSON)	Coefficients a, b of foot-point regression
samples	camera_id, frame, timestamp	INTEGER, INTEGER, REAL	Time reference
samples	track_id	INTEGER	Identity after re-linking
samples	x1, y1, x2, y2, score	REAL	Image box and confidence

Table	Field	Type	Description
samples	X, Y, speed, heading	REAL	Ground-plane state (m, m/s, rad)
samples	flags	INTEGER	Bit mask: interpolated, foot estimated, stopped
tracks	track_id, camera_id	INTEGER	Primary key, camera
tracks	t_start, t_end, length, straightness	REAL	Trajectory summary
tracks	entry_zone, exit_zone	INTEGER	Origin and destination zones
events	event_id, type, track_id, time, zone_or_line, direction	mixed	Crossings, stops, alerts

Table 5.3 Database schema of the trajectory store.

5.4.2 Configuration

All parameters are stored in a YAML configuration per camera, including the detector thresholds (high 0.6, low 0.1, birth 0.7), the association parameters (β, γ, gates), track life-cycle parameters (confirmation frames, maximum lost time in seconds), post-processing parameters (G_{max}, v_{max}, τ_{link}, smoothing window), the grid resolution, and the geometry of lines and zones in ground coordinates. Storing lines and zones in ground coordinates, rather than in pixels, means that they remain valid if the camera image is re-cropped or the resolution changes, provided the homography is updated.

5.4.3 JSON export

The JSON export contains one object per trajectory with the camera identifier, track identifier, summary fields and an array of samples; it is intended for exchange with GIS and visual analytics tools. Timestamps are written in ISO 8601 form, and coordinates are given in a local metric frame whose origin and orientation are documented in the camera record.

5.5 User Interface

The interface has a video panel with trajectory overlay, a bird's-eye panel with the heatmap or flow field, a statistics panel with counts and occupancy charts, and a configuration panel for thresholds, lines and zones (Figure 5.2).

[Figure 5.3 placeholder — insert diagram or result image here]

Figure 5.3 User interface of the system.

5.5.1 Calibration workflow

Calibration is a guided dialog. The user (i) selects a frame with a clearly visible ground; (ii) clicks at least four points whose ground coordinates are known (for example tile corners) and enters their coordinates; (iii) optionally clicks additional check points; (iv) inspects the projected metric grid and the reprojection error per region; and (v) saves the calibration. The dialog warns if the points are nearly collinear or cover only a small part of the image, since both lead to an ill-conditioned estimate.

[Figure 5.4 placeholder — insert diagram or result image here]

Figure 5.4 Calibration dialog with clicked correspondences and projected metric grid.

5.5.2 Interaction design

The views are linked: selecting a trajectory in the video panel highlights it in the bird's-eye panel and shows its speed profile; brushing a time interval in the counting chart filters all spatial views to that interval; and clicking a heatmap cell lists the trajectories that contributed to it, which allows the analyst to verify that a hotspot is caused by real pedestrians rather than by a false-positive detection such as a poster or mannequin. Lines and zones are drawn directly on either the video or the bird's-eye view and are converted to ground coordinates immediately. Colour maps and value ranges are fixed by default to allow comparison between videos, with a legend always shown.

5.5.3 Privacy features

Exported videos can be rendered with Gaussian blurring of the upper fifth of each box (the approximate head region) or with complete replacement of pedestrians by coloured

markers. In trajectory-only mode, no image data are stored and only anonymous identifiers are kept, which are not linked across days.

5.6 Chapter Summary

This chapter described the requirements, architecture, modules and threading model, implementation technologies, data schemas, configuration, and user interface of the pedestrian motion visualization system.

Chapter 6
Experiments and Analysis

Note to the author: this chapter defines the experiments. Cells marked [ ] must be filled with values from your own experiments on your GPU. Do not copy published numbers into the "ours" rows, and state clearly which baseline numbers you reproduced yourself and which are quoted from papers.

6.1 Datasets

- **MOT17**: pedestrian tracking benchmark with static and moving cameras; the half-split protocol (first half of each training sequence for tuning, second half for validation) is used for ablations, and the test set for final comparison.
- **MOT20**: very crowded scenes, used to test robustness under heavy occlusion.
- **Self-collected surveillance videos**: e.g., a campus entrance and a street crossing, each [ ] minutes long, recorded with a fixed camera. A homography is calibrated with measured ground points. For these videos, line-crossing counts and a subset of trajectories are annotated manually to evaluate visualization accuracy.

6.2 Experimental Environment

Item	Specification
GPU	[your GPU]
CPU / RAM	[ ]
Software	Python [ ], PyTorch [ ], CUDA [ ], TensorRT [ ]

Table 6.1 Experimental environment.

6.3 Comparison with State-of-the-Art Trackers

Tracker	HOTA	MOTA	IDF1	IDSW	Frag	FPS
SORT	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
DeepSORT	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
ByteTrack	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
OC-SORT	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
BoT-SORT	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
PMV-Track (ours)	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.2 Comparison on the MOT17 half-validation split using the same detector for all trackers.

Using the same detector for all trackers isolates the effect of the association algorithm. The same table should be produced for MOT20. Particular attention should be paid to IDF1, IDSW and Frag, which directly determine trajectory quality for visualization.

6.3.1 Results on MOT20

MOT20 contains scenes with up to several hundred pedestrians per frame, filmed from elevated viewpoints at night and in indoor halls, and is therefore closer to the dense surveillance scenarios that motivate crowd-safety visualization. Following common practice, the detector is fine-tuned on CrowdHuman and the MOT20 training set, and the tracker parameters are not re-tuned except for the birth threshold.

Tracker	HOTA	MOTA	IDF1	IDSW	Frag	FPS
SORT	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
ByteTrack	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
OC-SORT	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
BoT-SORT	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
StrongSORT	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
PMV-Track (ours)	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.3 Comparison on MOT20 (validation protocol or test server, to be stated) using the same detector for all trackers.

In MOT20 the adaptive ReID trigger is expected to be active for most detections, so the speed advantage over always-on ReID is expected to be smaller than on MOT17; the measured fraction ρ of detections passed to ReID should be reported for both datasets.

6.3.2 Per-sequence analysis

Aggregate metrics hide differences between scene types. The MOT17 sequences differ in camera motion (static versus moving), viewpoint (eye level versus elevated) and density, so per-sequence results show which component helps where. Results for the half-validation split are reported in the table below, with a short characterisation of each sequence.

Sequence	Camera	Viewpoint	Density	HOTA (ByteTrack)	HOTA (ours)	IDF1 (ByteTrack)	IDF1 (ours)	IDSW (ByteTrack)	IDSW (ours)
MOT17-02	Static	Eye level	Medium	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
MOT17-04	Static	Elevated	High	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
MOT17-05	Moving	Low	Low	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Sequence	Camera	Viewpoint	Density	HOTA (ByteTrack)	HOTA (ours)	IDF1 (ByteTrack)	IDF1 (ours)	IDSW (ByteTrack)	IDSW (ours)
MOT17-09	Static	Eye level	Low	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
MOT17-10	Moving	Eye level	Medium	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
MOT17-11	Moving	Low	Medium	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
MOT17-13	Moving	Elevated	Medium	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.4 Per-sequence results on the MOT17 half-validation split.

It is expected that CMC mainly benefits the moving-camera sequences (05, 10, 11, 13), and that the fused appearance cost mainly benefits the denser sequences (02, 04). The analysis should confirm or refute these expectations rather than assume them.

6.4 Ablation Study

Configuration	CMC	OC re-update	ReID (adaptive)	Direction term	Post-proc.	HOTA	IDF1	IDSW
Baseline (ByteTrack-style)						[ ]	[ ]	[ ]
+ CMC	✓					[ ]	[ ]	[ ]
+ OC re-update	✓	✓				[ ]	[ ]	[ ]
+ ReID fusion	✓	✓	✓			[ ]	[ ]	[ ]
+ Direction term	✓	✓	✓	✓		[ ]	[ ]	[ ]
+ Post-processing (full)	✓	✓	✓	✓	✓	[ ]	[ ]	[ ]

Table 6.5 Ablation study of PMV-Track on MOT17 half-validation.

Additionally compare: adaptive ReID versus always-on ReID (accuracy and FPS); linear versus Gaussian-process interpolation; and Savitzky–Golay versus moving-average smoothing.

ReID mode	Fraction ρ of detections embedded	HOTA	IDF1	IDSW	FPS
No ReID	0	[ ]	[ ]	[ ]	[ ]
Always-on ReID	1	[ ]	[ ]	[ ]	[ ]

ReID mode	Fraction ρ of detections embedded	HOTA	IDF1	IDSW	FPS
Adaptive ReID (ours)	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.6 Adaptive versus always-on appearance extraction on MOT17 half-validation.

6.4.1 Sensitivity to β and γ

The fused cost has two weights. β scales the appearance distance inside the minimum and γ weights the direction-consistency term. A grid search is performed on the MOT17 half-validation split, keeping all other parameters fixed, and HOTA and IDF1 are reported for each combination. A flat region around the chosen values indicates that the method does not depend on precise tuning.

β \ γ	0	0.1	0.2	0.3	0.5
0.5	[ ]	[ ]	[ ]	[ ]	[ ]
0.75	[ ]	[ ]	[ ]	[ ]	[ ]
1.0	[ ]	[ ]	[ ]	[ ]	[ ]
1.5	[ ]	[ ]	[ ]	[ ]	[ ]
2.0	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.7 Sensitivity of HOTA to β (rows) and γ (columns) on MOT17 half-validation. A second table of the same form should report IDF1.

The same procedure is applied to the direction window Δt (1, 3, 5 frames) and to the ReID trigger threshold θ_{occ} (0.1, 0.2, 0.3, 0.5), reporting accuracy and the fraction ρ.

6.4.2 Post-processing variants

Variant	Re-linking	Interpolation	Smoothing	HOTA	IDF1	IDSW	Frag	Speed RMSE (m/s)
None	–	–	–	[ ]	[ ]	[ ]	[ ]	[ ]
Linking only	Yes	–	–	[ ]	[ ]	[ ]	[ ]	[ ]
Linking + linear	Yes	Linear	–	[ ]	[ ]	[ ]	[ ]	[ ]
Linking + GP	Yes	GP	–	[ ]	[ ]	[ ]	[ ]	[ ]
Linking + GP + moving average	Yes	GP	Moving average	[ ]	[ ]	[ ]	[ ]	[ ]
Linking + GP + RTS	Yes	GP	RTS	[ ]	[ ]	[ ]	[ ]	[ ]
Linking + GP + Savitzky–Golay (full)	Yes	GP	Savitzky–Golay	[ ]	[ ]	[ ]	[ ]	[ ]

Variant	Re-linking	Interpolation	Smoothing	HOTA	IDF1	IDSW	Frag	Speed RMSE (m/s)
StrongSORT								
AFLink + GSI (reference)	AFLink	GSI	–	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.8 Comparison of post-processing variants. Speed RMSE is measured on the controlled walking sequences of this chapter.

The re-linking thresholds G_{max} (1, 2, 4 s) and τ_{link} (0.4, 0.6, 0.8) should also be varied, reporting the number of accepted links, the precision of accepted links (verified against ground-truth identities) and IDF1. Precision of links is important because a wrong link merges two people and is more harmful to visualization than a missing link.

6.5 Runtime Analysis

Module	Time per frame (ms)
Decoding and pre-processing	[ ]
Detection (TensorRT FP16)	[ ]
Camera motion compensation	[ ]
ReID (adaptive)	[ ]
Association and Kalman	[ ]
Rendering	[ ]
Total / FPS	[ ]

Table 6.9 Runtime breakdown at 1920 × 1080 input.

6.6 Evaluation of Visualization Outputs

6.6.1 Counting accuracy

On the self-collected videos, virtual-line counts produced by the system are compared with manual counts in 1-minute intervals. The mean absolute error (MAE) and the mean absolute percentage error (MAPE) are reported, comparing raw tracks with post-processed tracks to show that re-linking reduces double counting.

Video	Ground truth	Baseline count	PMV-Track count	MAPE (baseline)	MAPE (ours)
Campus entrance	[ ]	[ ]	[ ]	[ ]	[ ]
Street crossing	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.10 Line-crossing counting accuracy.

Counts are also reported per interval and per direction, because errors that cancel over a long period (a missed crossing in one minute and a double count in the next) are hidden

by totals. For each 1-minute interval i, with ground-truth count g_i and estimated count $\hat{c}_i$:

$$MAE = (1/n)\,\Sigma_i\,|\,\hat{c}_i - g_i\,|,\; MAPE = (100/n)\,\Sigma_i\,|\,\hat{c}_i - g_i\,|\,/\,g_i \qquad (6.1)$$

where intervals with $g_i = 0$ are excluded from MAPE.

Interval	Direction	Ground truth	Raw tracks	Post-processed tracks	Abs. error (raw)	Abs.error (post-proc.)
00:00–01:00	In	[ ]	[ ]	[ ]	[ ]	[ ]
00:00–01:00	Out	[ ]	[ ]	[ ]	[ ]	[ ]
01:00–02:00	In	[ ]	[ ]	[ ]	[ ]	[ ]
01:00–02:00	Out	[ ]	[ ]	[ ]	[ ]	[ ]
02:00–03:00	In	[ ]	[ ]	[ ]	[ ]	[ ]
02:00–03:00	Out	[ ]	[ ]	[ ]	[ ]	[ ]
...	...	...	...	...	...	...
Total / MAE	–	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.11 Line-crossing counts per 1-minute interval and direction for the campus entrance video (one such table per video, possibly in an appendix).

The counting errors should additionally be broken down by cause by inspecting each discrepancy: missed detections, identity switches leading to double counts, trajectories that end before the line, and hysteresis-related misses. This breakdown connects counting errors to specific tracker components.

6.6.2 Density and level-of-service evaluation

In scenes where congestion occurs, the instantaneous density estimate is compared with a manual count of persons inside a marked reference area of known size, sampled every 10 s. The agreement of the resulting Fruin level of service is reported as a confusion matrix.

Ground truth LoS \ Estimated LoS	A	B	C	D	E	F
A	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
B	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
C	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
D	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
E	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
F	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.12 Confusion matrix of the estimated Fruin level of service against manual counts in the reference area.

6.6.3 Homography calibration accuracy

Before evaluating metric statistics, the calibration of each self-collected video is validated. For each camera, n_{cal} calibration points and n_{chk} independent check points are measured with a tape measure or laser distance meter. The reprojection error (Section 4.2) is reported for calibration and check points, and separately for the near, middle and far thirds of the image. Plain DLT is compared with normalised DLT and with normalised DLT followed by non-linear refinement.

Video	Method	n_{cal}	n_{chk}	RMS calib. (m)	RMS check, near (m)	RMS check, middle (m)	RMS check, far (m)
Campus entrance	DLT (unnormalised)	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Campus entrance	Normalised DLT	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Campus entrance	Normalised DLT + LM	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Street crossing	DLT (unnormalised)	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Street crossing	Normalised DLT	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]
Street crossing	Normalised DLT + LM	[ ]	[ ]	[ ]	[ ]	[ ]	[ ]

Table 6.13 Homography reprojection error on calibration and held-out check points.

The effect of the foot-point estimation is evaluated on detections whose lower body is occluded: their estimated foot points are compared with manually annotated foot positions, and the ground-plane error is reported with and without the height regression.

Foot-point method	Mean ground error (m)	90th percentile error (m)
Bottom centre of box	[ ]	[ ]
Global height regression	[ ]	[ ]
Track-specific height	[ ]	[ ]

Table 6.14 Ground-plane localisation error for pedestrians with occluded feet.

6.6.4 Trajectory completeness and speed accuracy

Trajectory completeness is measured as the fraction of a person's ground-truth lifetime covered by a single predicted ID (similar to the "mostly tracked" ratio). Speed accuracy is measured by having volunteers walk known distances at a measured pace and comparing estimated with true speeds, which also validates the homography calibration.

6.6.5 Qualitative results

Present the trajectory overlay, speed-coloured trajectories, occupancy heatmap, flow field and dwell-time map for each video (Figures 6.1–6.3), and discuss the patterns revealed (main walking corridors, bottlenecks, gathering areas). Show how identity switches in the baseline tracker produce visible artefacts — such as broken trajectories, false "stops" and double counts — and how PMV-Track reduces them.

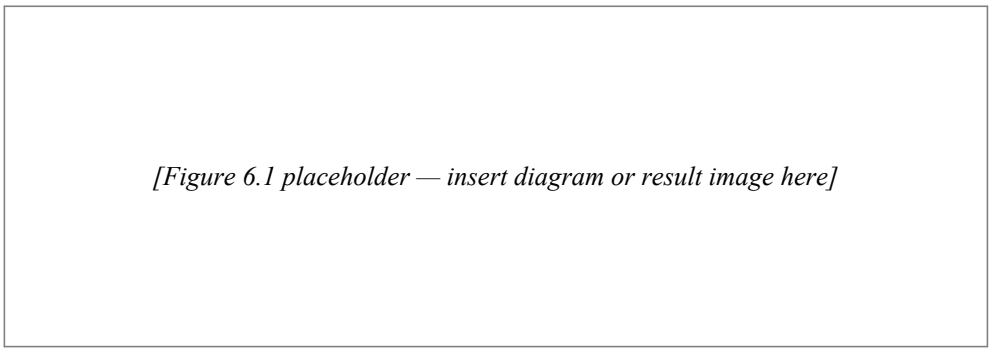


Figure 6.1 Trajectory and speed visualizations for the campus entrance video.

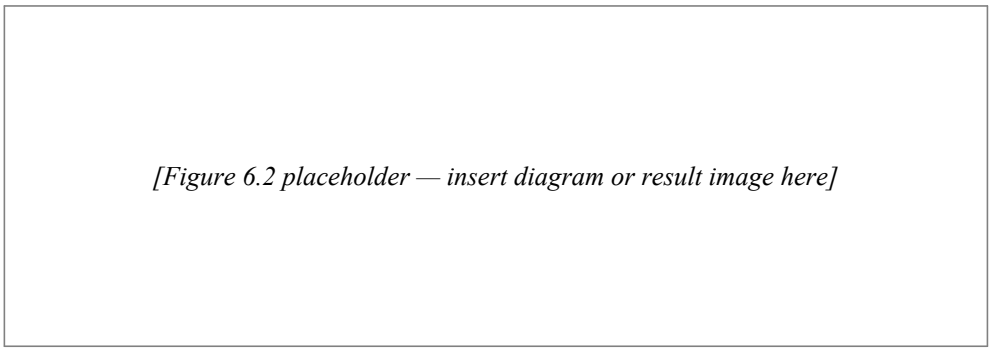


Figure 6.2 Occupancy heatmap and flow field on the bird's-eye view.

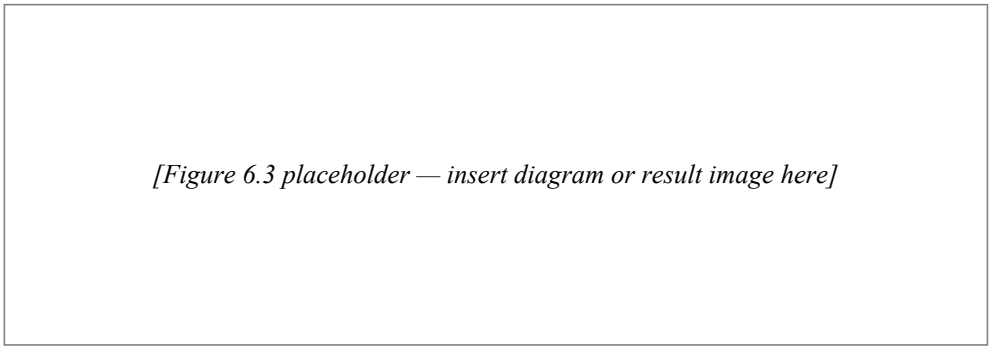


Figure 6.3 Comparison of visualization artefacts caused by identity switches (baseline versus PMV-Track).

6.6.6 User study

A small user study (e.g., 10–15 participants) can assess interpretability. Participants answer questions about pedestrian flow (e.g., "Which entrance is busiest?", "Where do people stop?") using either raw video or the visualizations. Task accuracy, completion time and a 5-point Likert rating of usefulness are recorded. Ethics approval and informed consent should be obtained according to your university's rules.

Design. A within-subject design is used: each participant performs comparable tasks in two conditions, (A) raw video with playback controls and (B) the visualization dashboard. To control learning effects, two different but comparable videos are used and the order of conditions and videos is counterbalanced. Before the study, participants receive a short training on the dashboard using a third video. Ground-truth answers are derived from the manual annotations.

Tasks. The tasks cover the levels of movement analysis distinguished by Andrienko et al. [24]: individual movers, spatial events and aggregated situations.

No.	Task	Answer type	Ground truth source
T1	Which entrance or exit is used most in the 10-minute clip?	Choice	Manual OD counts
T2	In which area do people most often stop or wait?	Region on map	Manual stop annotations
T3	What is the dominant walking direction along the main path?	Direction	Annotated trajectories
T4	During which minute is the area most crowded?	Time	Manual occupancy counts
T5	Is there any person walking against the main flow? If so, when?	Time / none	Manual inspection
T6	Estimate how many people crossed the marked line.	Number	Manual counts

Table 6.15 User study tasks.

Condition	Accuracy T1–T6	Mean completion time (s)	Mean Likert usefulness	SUS score
A: raw video	[ ]	[ ]	[ ]	[ ]
B: visualization dashboard	[ ]	[ ]	[ ]	[ ]

Table 6.16 User study results per condition.

Questionnaire. After each condition, participants rate the following statements on a 5-point Likert scale (1 = strongly disagree, 5 = strongly agree); after condition B they also complete the ten-item System Usability Scale (SUS) [65].

- Q1. I could easily see where people walk most.
- Q2. I could easily see where people stop.
- Q3. I could easily judge how crowded the area was at different times.
- Q4. I am confident that my answers are correct.
- Q5. The colours and symbols were easy to understand.
- Q6. I would use this tool to support decisions about the space.
- Q7. (Condition B only) I noticed visual artefacts that looked like tracking errors.

Accuracy is compared between conditions with McNemar's test per task or a paired test on the total score, completion time with a paired t-test or Wilcoxon signed-rank test depending on normality, and Likert ratings with the Wilcoxon signed-rank test. With 10–15 participants the study can reveal large effects only and should be reported as exploratory. Open comments are collected and summarised thematically.

6.7 Discussion

Discuss which components most improve the downstream visualizations, as opposed to benchmark metrics; the remaining failure modes (very dense crowds, long full occlusions, people with similar clothing, strong illumination changes); the effect of homography errors for non-planar scenes such as stairs; and the privacy implications of trajectory collection.

6.8 Chapter Summary

This chapter designed the experiments for tracking accuracy, ablation, runtime, counting accuracy, trajectory completeness and visualization quality.

Chapter 7
Conclusion and Future Work

7.1 Conclusions

- An effective pedestrian tracking algorithm, PMV-Track, was proposed, combining a viewpoint-adapted detector, camera motion compensation, observation-centric Kalman re-update, a fused IoU–appearance–direction association cost with adaptive ReID, and two-stage association of high- and low-score detections.
- A trajectory post-processing module with tracklet re-linking, Gaussian-process interpolation and smoothing was designed to produce complete trajectories suitable for visualization.
- A ground-plane projection method and motion feature extraction were designed so that speed, density and dwell time are measured in real-world units.
- Six complementary visualizations — fading trajectory overlays, speed/direction-coloured trajectories, occupancy heatmaps, flow fields, dwell-time maps and line-crossing statistics — were designed and integrated in a real-time system.
- A complete evaluation protocol covering tracking accuracy, efficiency, counting accuracy, trajectory quality and interpretability was designed.

7.2 Limitations

- The method assumes a single planar ground; multi-level scenes need multiple homographies or 3-D reconstruction.
- The system works on single cameras; pedestrians are not tracked across cameras.
- Extremely dense crowds (more than about 5 persons per m^2) remain difficult for detection-based tracking; density estimation methods may be more suitable there.

7.3 Future Work

- Multi-camera tracking with cross-camera re-identification and a unified site-level map.
- Integration of trajectory prediction models to visualize likely future congestion.
- Automatic abnormal behaviour detection (running, loitering, counter-flow) based on the extracted motion features.
- Privacy-preserving deployment, processing video on edge devices and transmitting only anonymous trajectories.

References

[1] Dalal N, Triggs B. Histograms of oriented gradients for human detection. CVPR, 2005.

[2] Ren S, He K, Girshick R, Sun J. Faster R-CNN: Towards real-time object detection with region proposal networks. NeurIPS, 2015.

[3] Ge Z, Liu S, Wang F, Li Z, Sun J. YOLOX: Exceeding YOLO series in 2021. arXiv:2107.08430, 2021.

[4] Shao S, Zhao Z, Li B, et al. CrowdHuman: A benchmark for detecting human in a crowd. arXiv:1805.00123, 2018.

[5] Zhang S, Benenson R, Schiele B. CityPersons: A diverse dataset for pedestrian detection. CVPR, 2017.

[6] Bewley A, Ge Z, Ott L, Ramos F, Upcroft B. Simple online and realtime tracking. IEEE ICIP, 2016.

[7] Wojke N, Bewley A, Paulus D. Simple online and realtime tracking with a deep association metric. IEEE ICIP, 2017.

[8] Wang Z, Zheng L, Liu Y, Li Y, Wang S. Towards real-time multi-object tracking. ECCV, 2020.

[9] Zhang Y, Wang C, Wang X, Zeng W, Liu W. FairMOT: On the fairness of detection and re-identification in multiple object tracking. IJCV, 2021.

[10] Zhang Y, Sun P, Jiang Y, et al. ByteTrack: Multi-object tracking by associating every detection box. ECCV, 2022.

[11] Cao J, Pang J, Weng X, Khirodkar R, Kitani K. Observation-centric SORT: Rethinking SORT for robust multi-object tracking. CVPR, 2023.

[12] Aharon N, Orfaig R, Bobrovsky B Z. BoT-SORT: Robust associations multi-pedestrian tracking. arXiv:2206.14651, 2022.

[13] Du Y, Zhao Z, Song Y, et al. StrongSORT: Make DeepSORT great again. IEEE Transactions on Multimedia, 2023.

[14] Maggiolino G, Ahmad A, Cao J, Kitani K. Deep OC-SORT: Multi-pedestrian tracking by adaptive re-identification. IEEE ICIP, 2023.

[15] Zhou K, Yang Y, Cavallaro A, Xiang T. Omni-scale feature learning for person re-identification. ICCV, 2019.

[16] Meinhardt T, Kirillov A, Leal-Taixé L, Feichtenhofer C. TrackFormer: Multi-object tracking with transformers. CVPR, 2022.

[17] Zeng F, Dong B, Zhang Y, et al. MOTR: End-to-end multiple-object tracking with transformer. ECCV, 2022.

[18] Luiten J, Osep A, Dendorfer P, et al. HOTA: A higher order metric for evaluating multi-object tracking. IJCV, 2021.

[19] Milan A, Leal-Taixé L, Reid I, Roth S, Schindler K. MOT16: A benchmark for multi-object tracking. arXiv:1603.00831, 2016.

[20] Dendorfer P, Rezatofighi H, Milan A, et al. MOT20: A benchmark for multi object tracking in crowded scenes. arXiv:2003.09003, 2020.

[21] Hartley R, Zisserman A. Multiple View Geometry in Computer Vision (2nd ed.). Cambridge University Press, 2004.

[22] Alahi A, Goel K, Ramanathan V, et al. Social LSTM: Human trajectory prediction in crowded spaces. CVPR, 2016.

[23] Gupta A, Johnson J, Fei-Fei L, Savarese S, Alahi A. Social GAN: Socially acceptable trajectories with generative adversarial networks. CVPR, 2018.

[24] Andrienko G, Andrienko N, Bak P, Keim D, Wrobel S. Visual Analytics of Movement. Springer, 2013.

[25] Savitzky A, Golay M J E. Smoothing and differentiation of data by simplified least squares procedures. Analytical Chemistry, 1964.

[26] Fruin J J. Pedestrian Planning and Design. Metropolitan Association of Urban Designers and Environmental Planners, 1971.

[27] Lucas B D, Kanade T. An iterative image registration technique with an application to stereo vision. IJCAI, 1981.

[28] Fischler M A, Bolles R C. Random sample consensus. Communications of the ACM, 1981.

[29] Jonker R, Volgenant A. A shortest augmenting path algorithm for dense and sparse linear assignment problems. Computing, 1987.

[30] Felzenszwalb P F, Girshick R B, McAllester D, Ramanan D. Object detection with discriminatively trained part-based models. IEEE TPAMI, 2010.

[31] Redmon J, Divvala S, Girshick R, Farhadi A. You only look once: Unified, real-time object detection. CVPR, 2016.

[32] Liu W, Anguelov D, Erhan D, et al. SSD: Single shot multibox detector. ECCV, 2016.

[33] Tian Z, Shen C, Chen H, He T. FCOS: Fully convolutional one-stage object detection. ICCV, 2019.

[34] Lin T Y, Goyal P, Girshick R, He K, Dollár P. Focal loss for dense object detection. ICCV, 2017.

[35] Wang X, Xiao T, Jiang Y, Shao S, Sun J, Shen C. Repulsion loss: Detecting pedestrians in a crowd. CVPR, 2018.

[36] Chu X, Zheng A, Zhang X, Sun J. Detection in crowded scenes: One proposal, multiple predictions. CVPR, 2020.

[37] Liu S, Huang D, Wang Y. Adaptive NMS: Refining pedestrian detection in a crowd. CVPR, 2019.

[38] Bodla N, Singh B, Chellappa R, Davis L S. Soft-NMS: Improving object detection with one line of code. ICCV, 2017.

[39] Zhang L, Li Y, Nevatia R. Global data association for multi-object tracking using network flows. CVPR, 2008.

[40] Reid D B. An algorithm for tracking multiple targets. IEEE Transactions on Automatic Control, 1979.

[41] Bergmann P, Meinhardt T, Leal-Taixé L. Tracking without bells and whistles. ICCV, 2019.

[42] Zhou X, Koltun V, Krähenbühl P. Tracking objects as points. ECCV, 2020.

[43] Sun P, Cao J, Jiang Y, et al. TransTrack: Multiple object tracking with transformer. arXiv:2012.15460, 2020.

[44] Luo H, Gu Y, Liao X, Lai S, Jiang W. Bag of tricks and a strong baseline for deep person re-identification. CVPR Workshops, 2019.

[45] He S, Luo H, Wang P, Wang F, Li H, Jiang W. TransReID: Transformer-based object re-identification. ICCV, 2021.

[46] Hermans A, Beyer L, Leibe B. In defense of the triplet loss for person re-identification. arXiv:1703.07737, 2017.

[47] Zhang Y, Zhou D, Chen S, Gao S, Ma Y. Single-image crowd counting via multi-column convolutional neural network. CVPR, 2016.

[48] Li Y, Zhang X, Chen D. CSRNet: Dilated convolutional neural networks for understanding the highly congested scenes. CVPR, 2018.

[49] Helbing D, Molnár P. Social force model for pedestrian dynamics. Physical Review E, 1995.

[50] Mehran R, Oyama A, Shah M. Abnormal crowd behavior detection using social force model. CVPR, 2009.

[51] Mahadevan V, Li W, Bhalodia V, Vasconcelos N. Anomaly detection in crowded scenes. CVPR, 2010.

[52] Salzmann T, Ivanovic B, Chakravarty P, Pavone M. Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data. ECCV, 2020.

[53] Hägerstrand T. What about people in regional science? Papers of the Regional Science Association, 1970.

[54] Kraak M J. The space-time cube revisited from a geovisualization perspective. Proceedings of the 21st International Cartographic Conference, 2003.

[55] Silverman B W. Density Estimation for Statistics and Data Analysis. Chapman and Hall, 1986.

[56] Rasmussen C E, Williams C K I. Gaussian Processes for Machine Learning. MIT Press, 2006.

[57] Rauch H E, Tung F, Striebel C T. Maximum likelihood estimates of linear dynamic systems. AIAA Journal, 1965.

[58] Kalman R E. A new approach to linear filtering and prediction problems. Journal of Basic Engineering, 1960.

[59] Kuhn H W. The Hungarian method for the assignment problem. Naval Research Logistics Quarterly, 1955.

[60] Hartley R I. In defense of the eight-point algorithm. IEEE TPAMI, 1997.

[61] Shi J, Tomasi C. Good features to track. CVPR, 1994.

[62] Bernardin K, Stiefelhagen R. Evaluating multiple object tracking performance: The CLEAR MOT metrics. EURASIP Journal on Image and Video Processing, 2008.

[63] Ristani E, Solera F, Zou R, Cucchiara R, Tomasi C. Performance measures and a data set for multi-target, multi-camera tracking. ECCV Workshops, 2016.

[64] Pellegrini S, Ess A, Schindler K, Van Gool L. You'll never walk alone: Modeling social behavior for multi-target tracking. ICCV, 2009.

[65] Brooke J. SUS: A quick and dirty usability scale. In: Usability Evaluation in Industry, Taylor and Francis, 1996.

[66] Holten D, van Wijk J J. Force-directed edge bundling for graph visualization. Computer Graphics Forum, 2009.

[67] Lin T Y, Maire M, Belongie S, et al. Microsoft COCO: Common objects in context. ECCV, 2014.

[68] Zhang Y, Wang T, Zhang X. MOTRv2: Bootstrapping end-to-end multi-object tracking by pretrained object detectors. CVPR, 2023.

Acknowledgements

[Write your acknowledgements here.]